



第四届eBPF开发者大会

基于eBPF的端到端请求追踪

从云原生到多智能体

陈鹏飞

中山大学 计算机学院

2026年4月

报告提纲

01

研究背景与动机

云原生可观测性挑战与eBPF机遇

02

零侵入的HTTP请求追踪

如何在完全不改代码的情况下追踪微服务请求

03

容器健康状态的精准感知

链路追好了，但容器本身是否健康

04

突破协议限制：多协议精确追踪

gRPC/Thrift无处不在，如何平衡精度与开销

05

向上延伸：GPU计算图的可观测性

大模型时代，GPU上的计算模式谁来追踪

06

AI时代：多智能体系统追踪

Agent调用Agent，工具调用工具——这条链路怎么追

07

总结与展望

eBPF统一可观测性基础设施的未来

发表成果

HTTP追踪

TPDS 2025

容器探测

TSC 2025

多协议追踪

CoNEXT 2026

GPU追踪

IWQoS 2025

Agent追踪

ASE 2026投稿



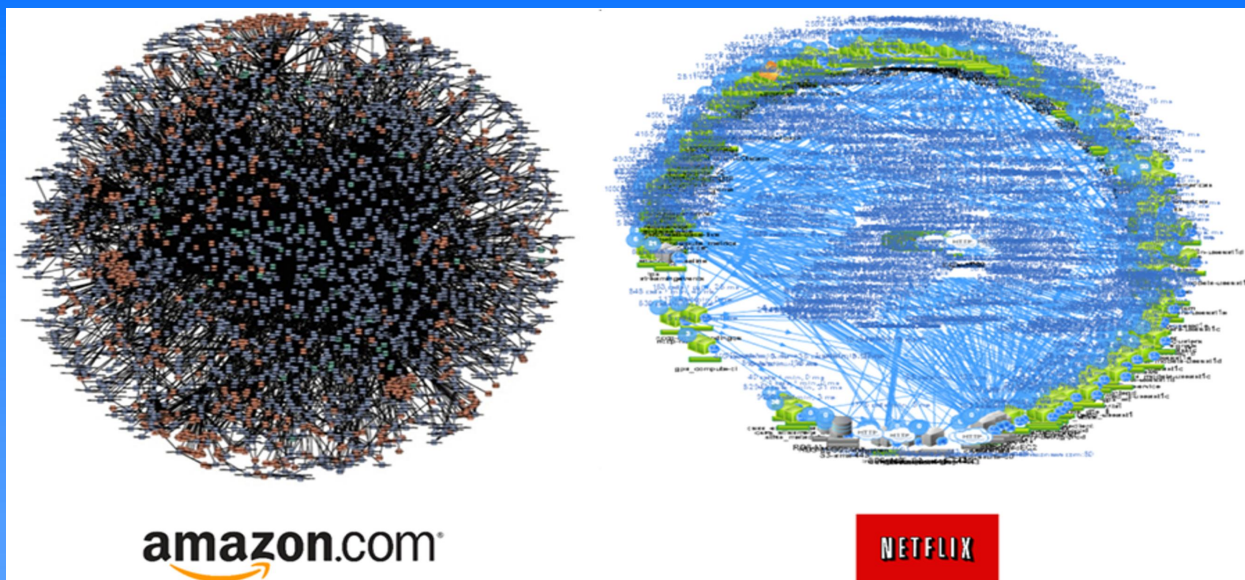
01 研究背景与动机

云原生系统可观测性的挑战与eBPF机遇

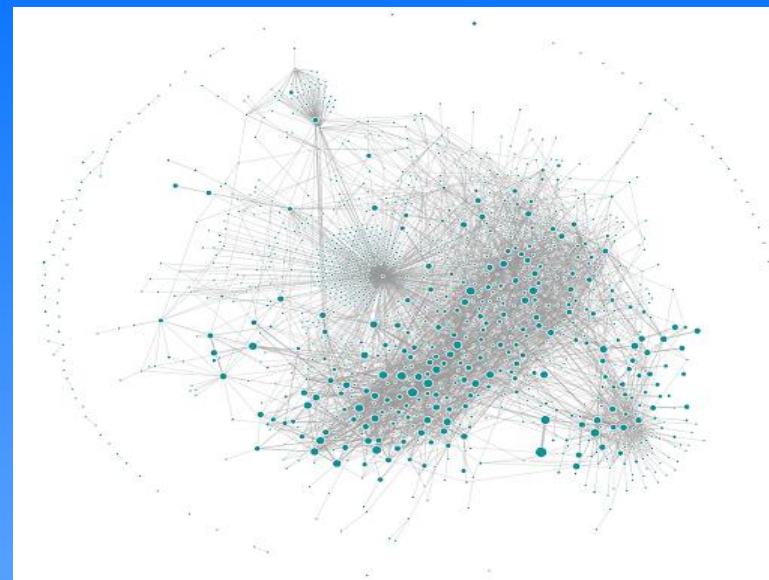
"凌晨3点，线上告警响了，一个请求跨越23个微服务——你如何在5分钟内定位根因？"

云原生：规模与复杂性

数千微服务 · 跨语言跨框架 · 持续交付 · 容器短生命周期



Amazon (左) 与 Netflix (右) 的微服务调用拓扑



典型微服务系统的调用关系图

规模爆炸

WeChat 3000+ 微服务
Netflix 700+, Uber
2200+
调用链跨越数十个服务

高度动态

持续集成/持续交付
容器生命周期以分钟计
运维算法需实时适应

技术多样

Golang/Java/Python/Node
Dubbo/gRPC/Spring Cloud
AWS/阿里云/私有云

故障频发

CNCF调查: 50%团队
每周遇到生产事故
定位耗时占运维60%+

Metrics / Logs / Traces — 为什么Trace是微服务诊断的关键？

The image displays three Grafana dashboards monitoring server performance. The top-left dashboard, titled 'server requests', shows the number of requests per second for four web servers (web_server_01, web_server_02, web_server_03, and web_server_04) over a period from 13:30 to 15:20. The top-right dashboard, titled 'Memory / CPU', shows memory usage (in GB) and CPU load (in %) for the same period. The bottom dashboard, titled 'Traffic In/Out', shows network traffic (in MB/s) for various interfaces (upper_25, upper_50, upper_75, upper_90, upper_95, rpu1, and rpu2) over the same period. Each dashboard includes a legend and a time range selector.

聚合数值 · 仪表盘 CPU/QPS/延迟等趋势 告诉你"出了问题"

[Dashboard / Filebeat PostgresSQL Overview](#)
 Search... (e.g. type:200 AND extension:PHP)

[Full screen](#)
[Share](#)
[Clone](#)
[Edit](#)
[Reporting](#)
[Auto-refresh](#)
Last 10 minutes

[Options](#)

Add a filter +

All Logs [Filebeat PostgresSQL]

Time	Type	Host	Source	Message	Duration	Action
November 27th 2018, 15:37:10.992	elastic	opbeans	LOG	duration: 0.435 ms bind 5, 14: select product_id as col_0, product_name as col_1, product_name as col_2, product_stock as col_3, sum(order_line_amount) as col_4, from order_lines order_line0, left outer join products product1 on	-	
November 27th 2018, 15:37:10.992	elastic	opbeans	DETAIL	parameters: \$1 = "3"	-	
November 27th 2018, 15:37:10.992	elastic	opbeans	LOG	duration: 0.375 ms execute 5, 14: select product_id as col_0, product_name as col_1, product_name as col_2, product_stock as col_3, sum(order_line_amount) as col_4, from order_lines order_line0, left outer join products product1 on	-	
November 27th 2018, 15:37:10.991	elastic	opbeans	LOG	duration: 0.031 ms bind 5, 1:	-	
November 27th 2018, 15:37:10.669	elastic	opbeans	LOG	parameters: \$1 = "603"	-	
November 27th 2018, 15:37:10.669	elastic	opbeans	LOG	duration: 0.048 ms execute runnamed=SELECT	-	
November 27th 2018, 15:37:10.641	elastic	opbeans	DETAIL	parameters: \$1 = "1", \$2 = "1000"	-	
November 27th 2018, 15:37:10.640	elastic	opbeans	LOG	duration: 2.326 ms bind runnamed=SELECT c FROM customers c LEFT JOIN orders o ON c.id=o.customer, left JOIN order_lines l ON o.id=l.order, left JOIN	-	

离散事件·全文搜索 错误堆栈/请求详情 告诉你"发生了什么"

[illegible]

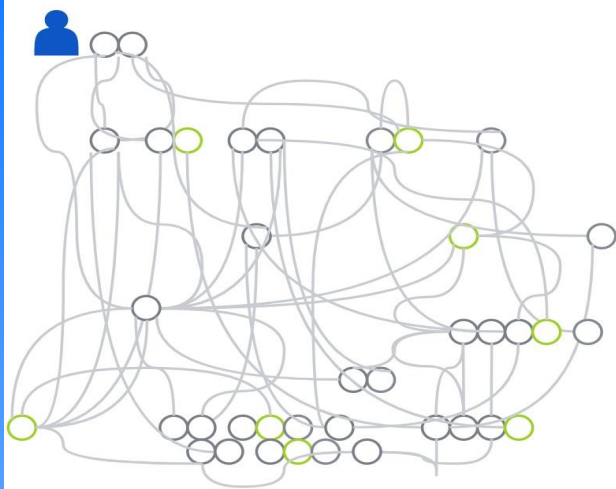
因果链路 · 端到端
请求在服务间的完整路径
告诉你“为什么慢/为什么错”

本报告聚焦

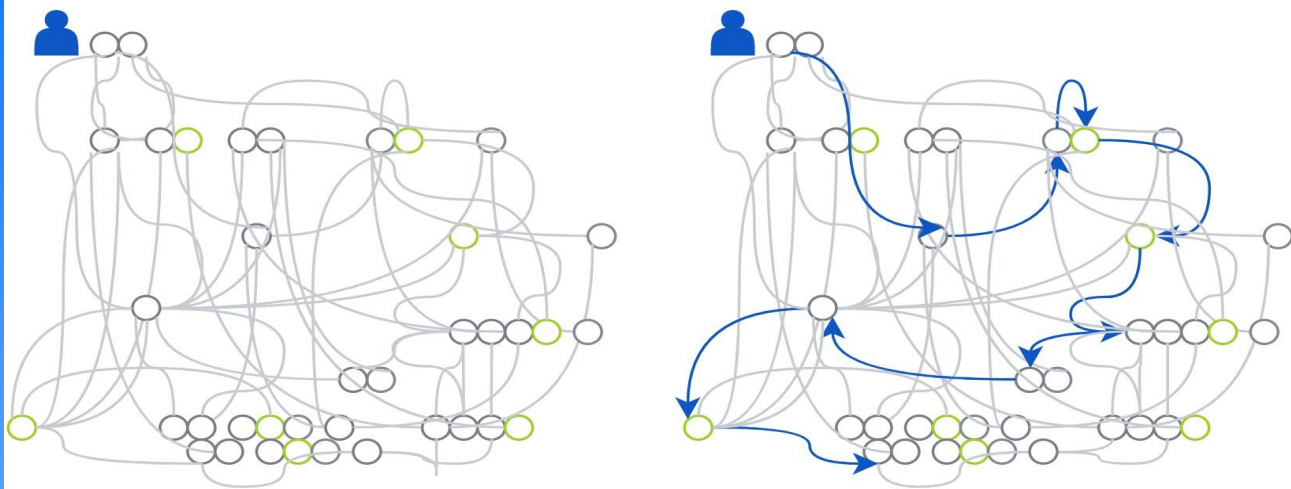
端到端请求追踪：从混沌到可见

在复杂微服务调用中追踪一个请求的完整路径

No Distributed Tracing



Distributed Tracing

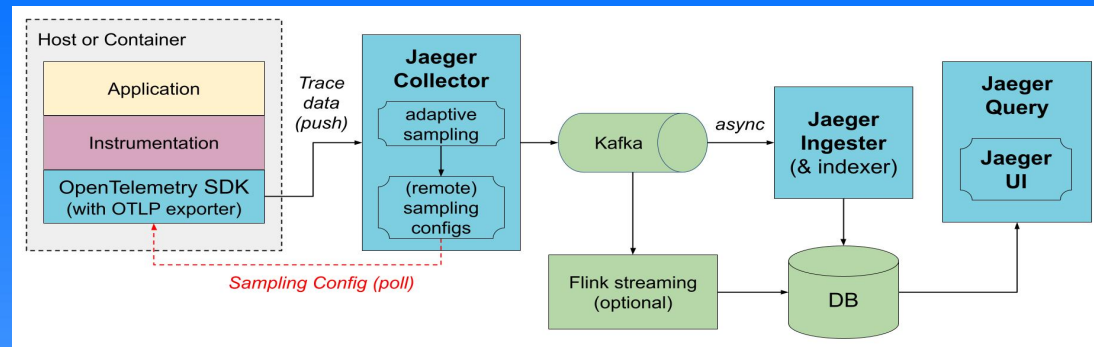


—— service-to-service connection
—— individual request path

没有追踪(左): 只看到连接, 不知道请求走向 vs 有追踪(右): 请求路径清晰可见

核心问题

能否实现完全不修改应用代码的端到端请求追踪?
eBPF提供了在内核层面解决这一问题的可能性。



典型追踪系统架构 (Jaeger): SDK埋点 → Collector → Storage → Query

当前主流方案的代价

- ▶ 需要在每个服务中集成SDK (OpenTelemetry等)
- ▶ 修改业务代码注入Trace Context
- ▶ 跨语言/跨框架需适配不同的instrumentation库
- ▶ 第三方库/中间件覆盖困难 (线程池、异步等)
- ▶ 升级维护成本高: 每次SDK更新需全量重新部署

云原生系统的可观测性挑战



微服务复杂交互

单次请求可能跨越数十个服务，故障传播路径复杂



代码侵入性高

传统追踪(*Zipkin/Jaeger/OTel*)需修改应用代码，维护成本极高



追踪不准确

非侵入方案在多线程、异步、*RPC*场景下准确率大幅下降



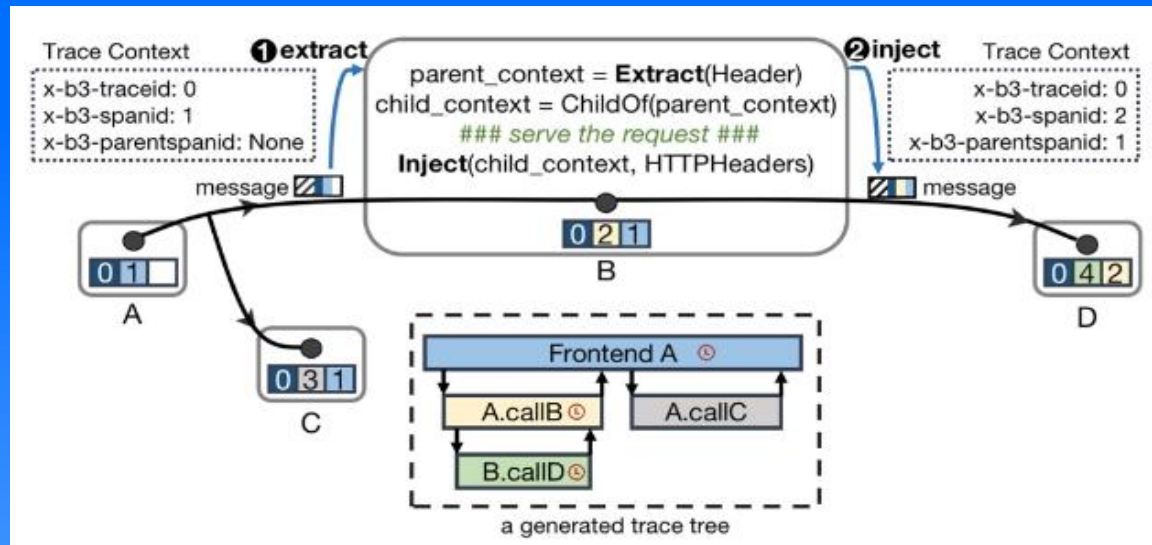
探针信息不足

*K8s*探针仅检测连接/响应，无法感知容器内部应用状态



AI系统新挑战

*ML*训练推理、多智能体系统缺乏专用可观测性方案



端到端追踪研究整体框架

五项系统性工作

- HTTP追踪** 零侵入请求级分布式追踪 TPDS'25
- 容器探测** *eBPF*增强*K8s*健康感知 TSC'25
- 多协议追踪** 内核级精确追踪+采样 CoNEXT'26
- GPU追踪** *ML*系统计算图追踪 IWQoS'25
- Agent追踪** 多智能体链路追踪 ASE'26投稿

eBPF: 内核可编程的革命性技术

什么是eBPF?

- ▶ 扩展的Berkeley Packet Filter, Linux内核中的沙箱虚拟机
- ▶ 无需修改内核源码或加载内核模块即可扩展内核功能
- ▶ 程序可附着于kprobe/tracepoint/TC/XDP等挂载点
- ▶ 验证器(Verifier)保证程序安全性, JIT编译保证性能
- ▶ 广泛应用于网络(Cilium)、安全、可观测性领域

零侵入

无需修改应用代码运行时动态加载

高性能

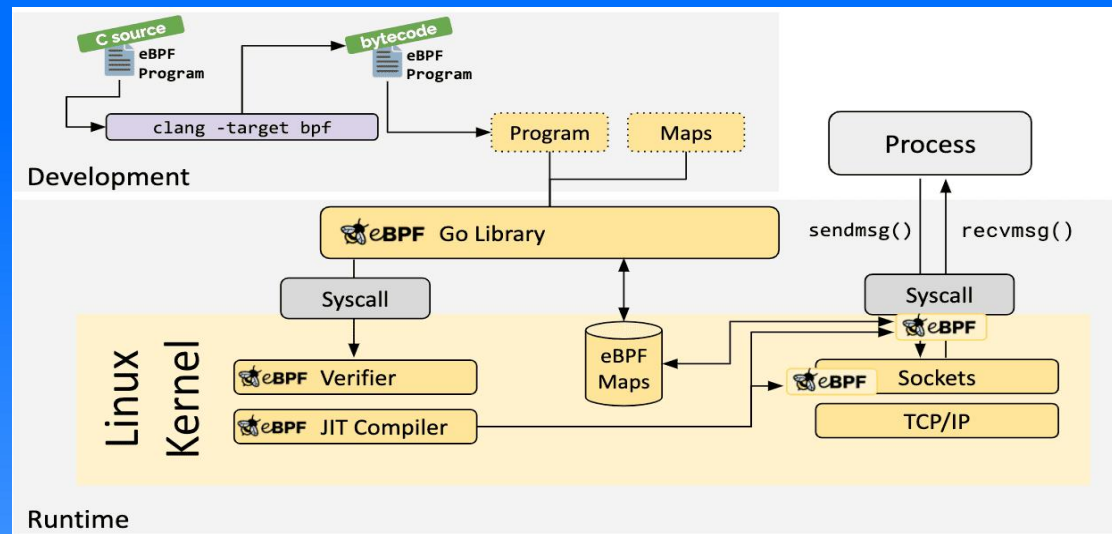
内核态运行
JIT编译优化

安全可靠

验证器静态检查
沙箱隔离执行

灵活通用

动态加载卸载
支持多种挂载点



eBPF在Linux内核中的工作原理

我们的eBPF追踪研究路线

Stage 1

HTTP请求零侵入追踪

Stage 2

容器健康状态感知

Stage 3

多协议精确追踪+采样

Stage 4

GPU计算图追踪

Stage 5

多智能体系统追踪

02 零侵入的HTTP请求追踪

能否在完全不修改代码的情况下, 实现精确的分布式追踪?

eBPF内核态追踪生成器

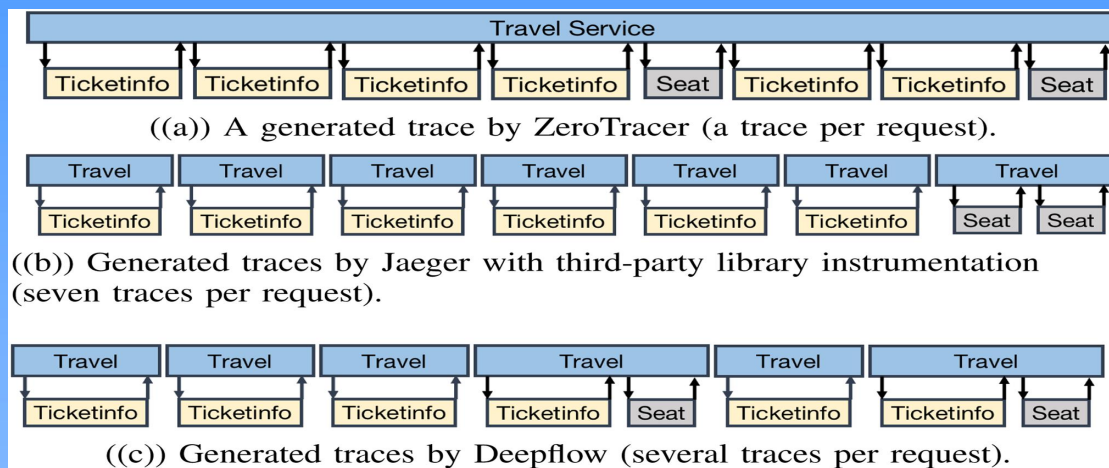
现有方案的局限性

从侵入式到非侵入式: 追踪系统的能力对比

方案类别	代表系统	代码侵入	多线程支持	采样控制	准确率	维护成本
侵入式	Zipkin/Jaeger/OTel	需要修改	支持	支持	高	高
非侵入-统计	Vpath/MicroRank	无需修改	不支持	不支持	中	低
非侵入-eBPF	DeepFlow	无需修改	不支持	有限	低~中	低
ZeroTracer	本文	无需修改	支持	支持	91%+	极低

现有非侵入追踪的关键不足

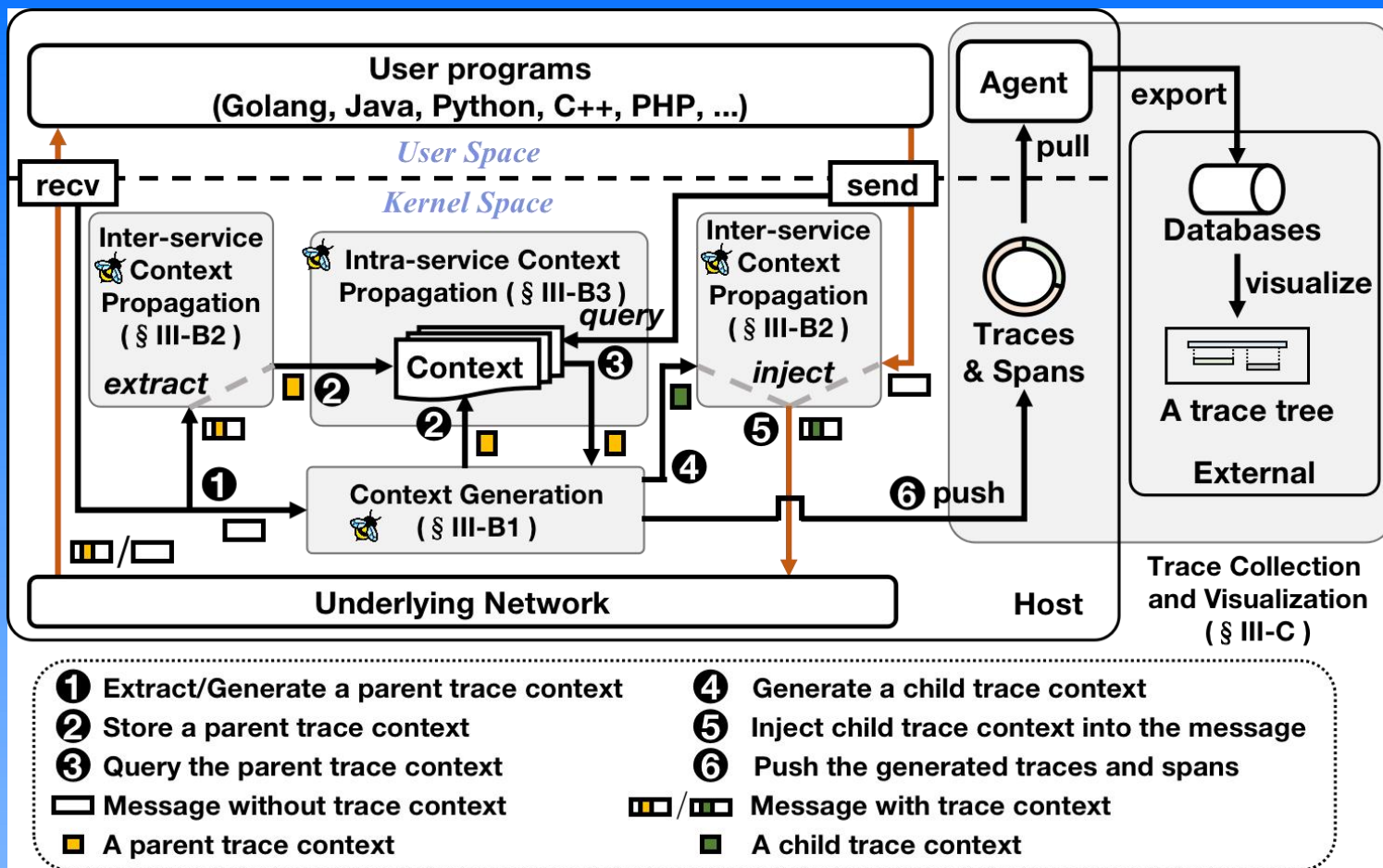
- ▶ 仅作为Monitor/Span Generator, 无法在内核生成完整Trace
- ▶ 不支持多线程微服务应用(如线程池、异步处理模型)
- ▶ 无法实现Trace级采样, 只能全量采集或随机丢弃
- ▶ 缺乏在线追踪能力, 依赖离线日志分析重建因果关系



分布式追踪Trace Tree示例

内核态在线追踪生成器架构

首个在eBPF内核中直接生成完整Trace的系统



内核态追踪生成器整体架构 (TPDS论文): eBPF Trace Generator + Context Propagation

核心设计

eBPF Trace Generator

内核态在线追踪生成, 区别于现有 Monitor/Span Generator 方案

HTTP头部解析

在TC层拦截数据包, 解析HTTP请求/响应头部元数据

Context注入/提取

在HTTP Header中注入32字节自定义 Trace Context

因果关系推断

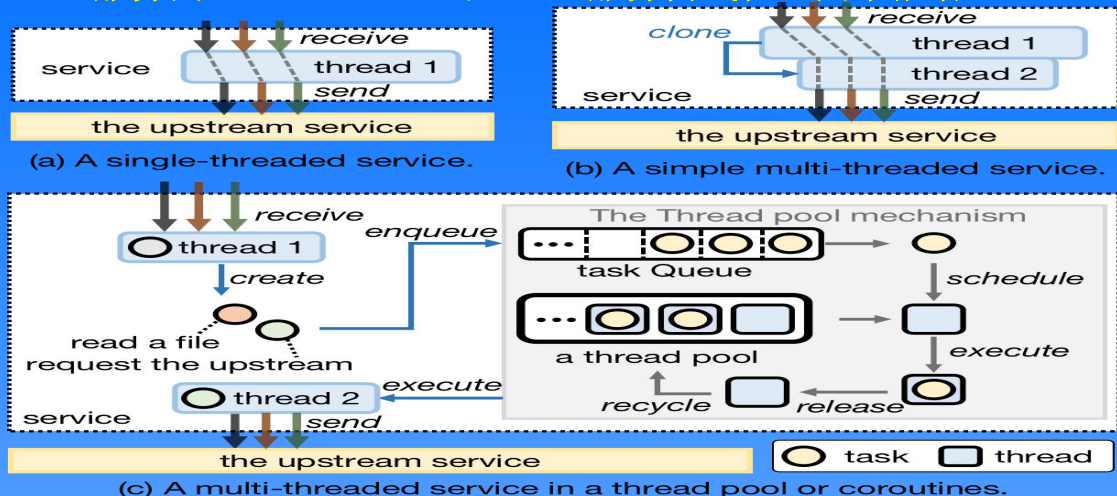
基于线程模型自动发现parent-child关系

OpenTelemetry兼容

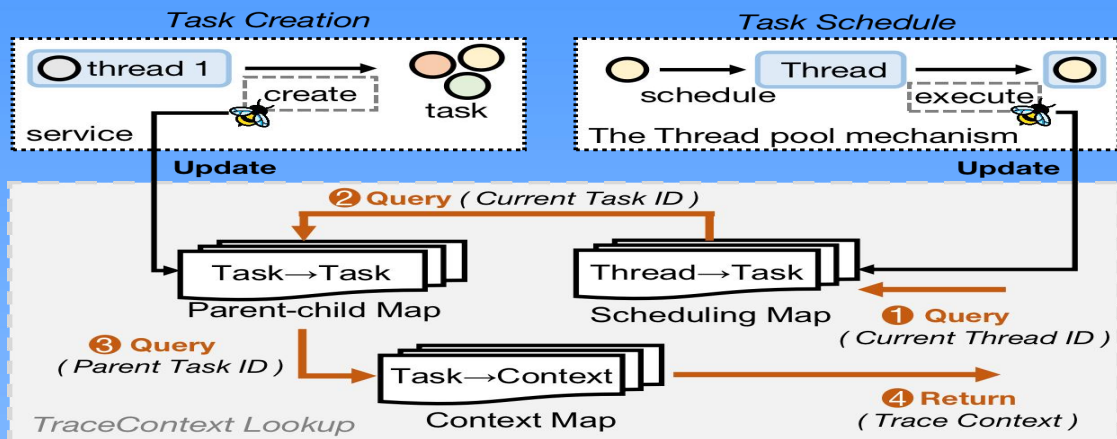
生成标准Trace ID(16B) + Span ID(8B), 对接现有后端

三种处理模型与上下文传播

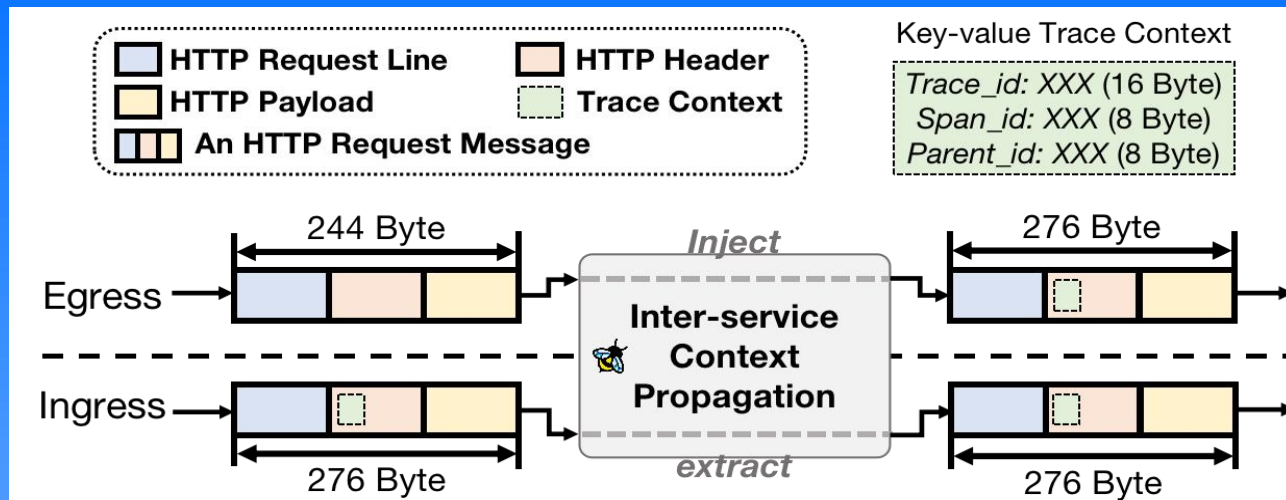
服务间HTTP Header注入 + 服务内线程池因果推断



三种微服务请求处理模型 (Sequential/Thread-Pool/Async)



2026年4月 · 中国 线程池场景下的上下文传播解决方案



服务间传播: HTTP Header中注入Trace Context (32字节)

Sequential Model

单线程串行, 按时间顺序建立因果关系

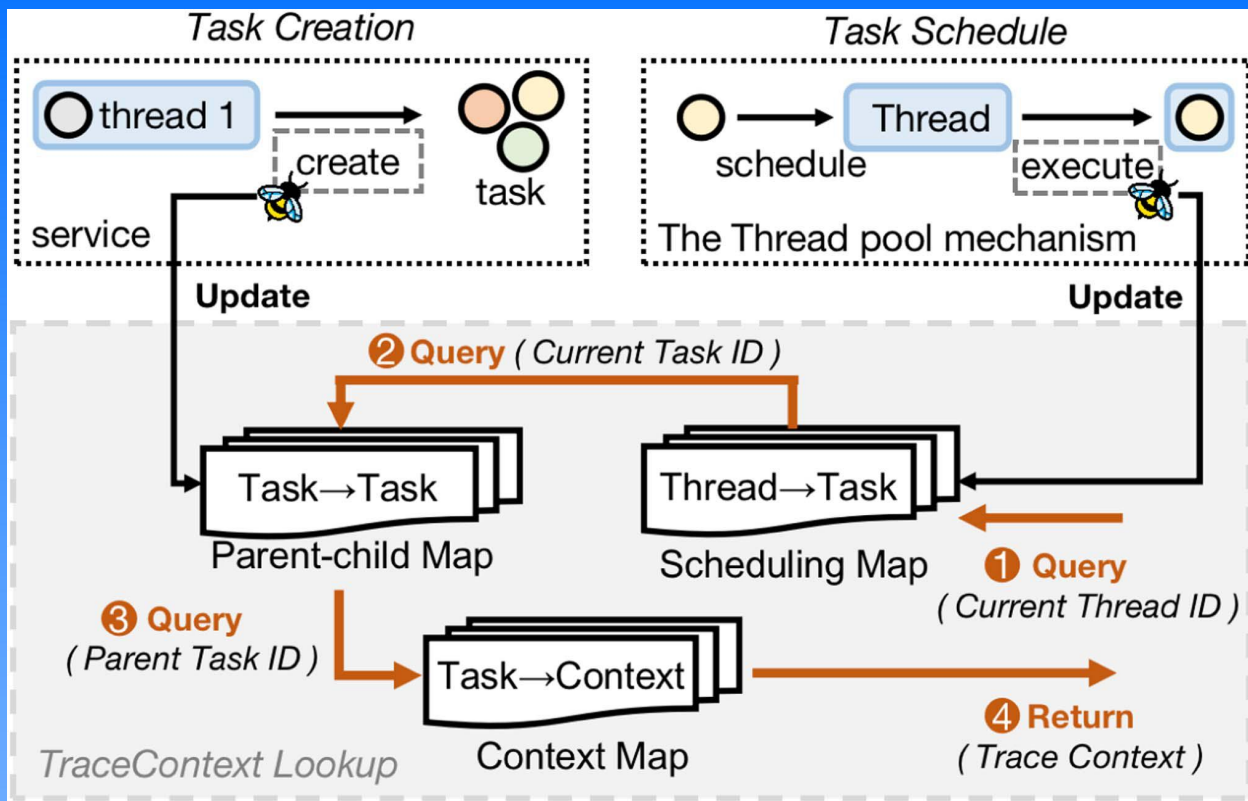
Thread-Pool Model

线程池并行, 通过thread-id追踪请求路由 (Java典型)

Asynchronous Model

异步事件驱动, 通过epoll事件关联 (Nginx/Node)

Trace Context设计与生成流程



内核态追踪生成流程

Trace ID 格式 (16字节)

Host IP	32 bit	主机IP地址
Timestamp	52 bit	请求进入时间戳
Seq Num	16 bit	请求序列号
Debug Tag	4 bit	调试标记
PID	24 bit	进程ID

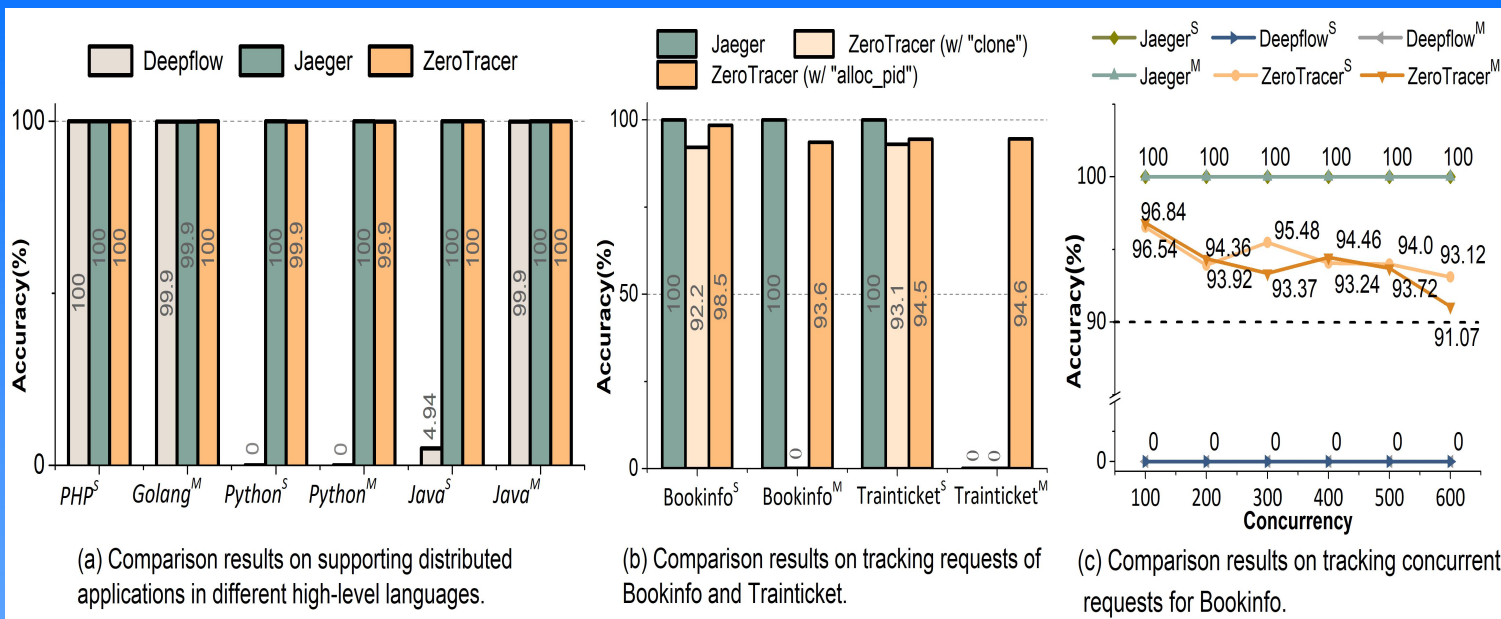
Span ID 格式 (8字节)

Client IP	16 bit	客户端IP后16位
Server IP	16 bit	服务端IP后16位
Seq Num	32 bit	Span在Trace中的序号

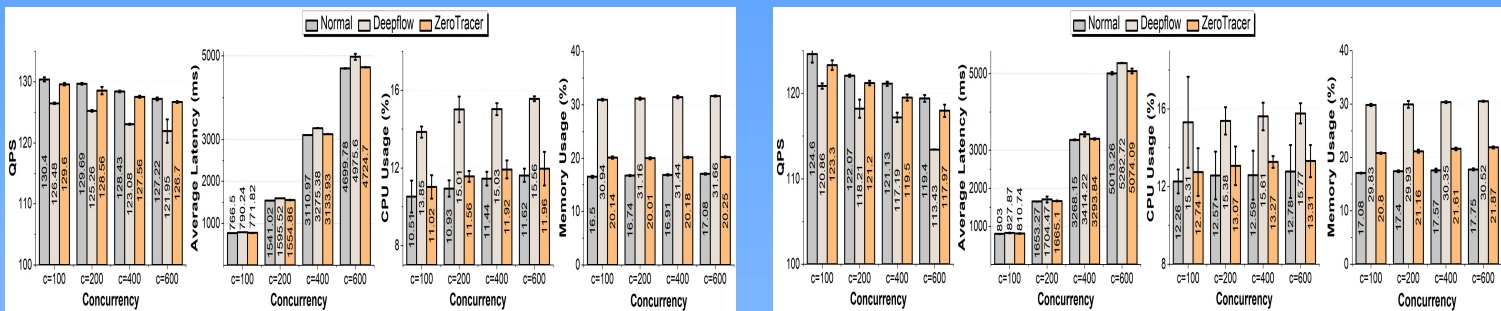
关键创新: 仅增加32字节HTTP头, 即可在内核实现完整的Trace上下文传播

零侵入追踪的实验评估

Train-Ticket + Sock-Shop + Bookinfo 基准测试



多语言/微服务基准/并发影响: Jaeger vs DeepFlow vs ZeroTracer



2026年4月 中国

多线程Bookinfo开销

91%+
追踪准确率
稳定高并发场景

0.5%-1.2%
延迟增加
几乎无感知

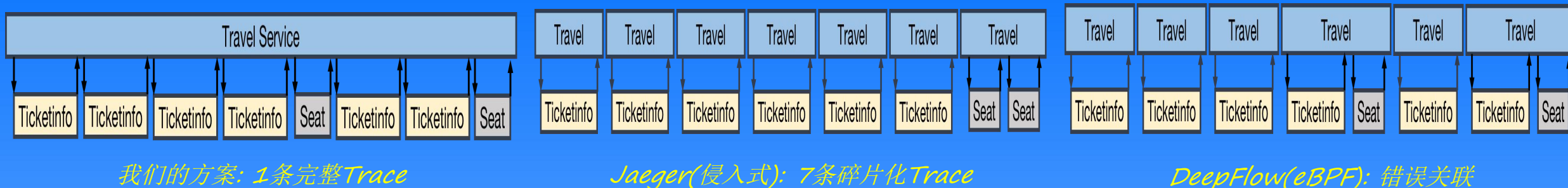
3%-5.8%
CPU/内存额外消耗
资源友好

首个

支持多线程微服务的
非侵入eBPF追踪系统

定性对比: 多线程场景Trace完整性

同一请求, 三种方案生成的Trace有何不同? · 阿里云生产部署验证



多线程场景下的本质差异

- ▶ Jaeger: 第三方库无法覆盖自定义线程池, 一个请求被拆成7条碎片trace
- ▶ DeepFlow: 仅做Span生成, 无法在线推断多线程因果关系, trace错误
- ▶ ZeroTracer: 内核线程模型 + 32字节HTTP头, 生成唯一完整trace

生产部署验证

在阿里云真实Tracing Console中部署验证, trace结构、span时序、服务拓扑可视化完整呈现, 与商业追踪方案等同质量

Span Name	Timeline (ms)
▼ server:productpage_pod->NONE ①	10.427ms
▼ client:productpage_pod->detail_svc ②	0.894ms
server:detail_pod->productpage_pod ③	0.585ms
▼ client:productpage_pod->review_svc ④	0.66ms
server:review1_pod->productpage_pod ⑤	0.321ms

阿里云 Tracing Console 中ZeroTracer生成的Bookinfo Trace¹⁵

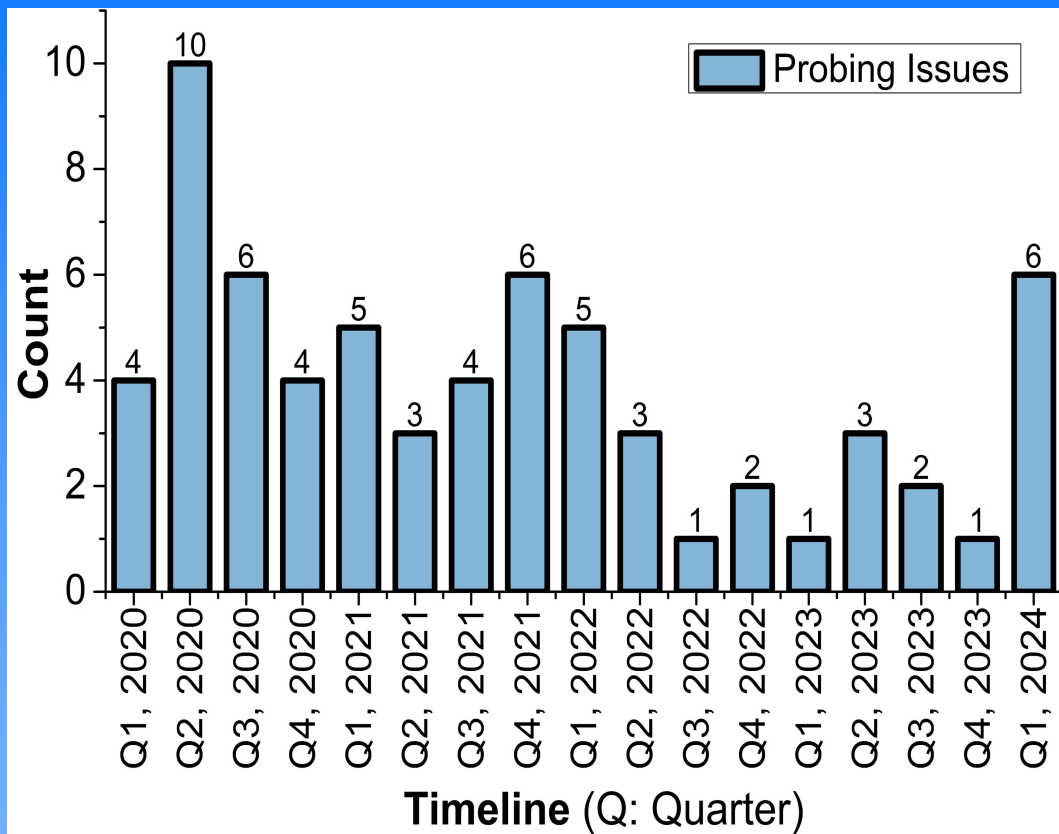
03 容器健康状态的精准感知

链路追踪解决了请求可见性, 但容器本身是否健康?

*K8s*探针给不了你真正的答案, *eBPF*可以

K8s探针的问题分析

2020年以来Kubernetes/Istio上68个探针相关Issue分析



K8s/Istio探针相关Issue分布

K8s探针三大问题



探测信息不足

HTTP探针仅检查连接是否成功
TCP探针仅验证端口可达
无法感知应用内部状态



探测延迟与不准确

探针周期性执行(如每10秒)
容器已异常但探针尚未触发
超时设置不当导致误判

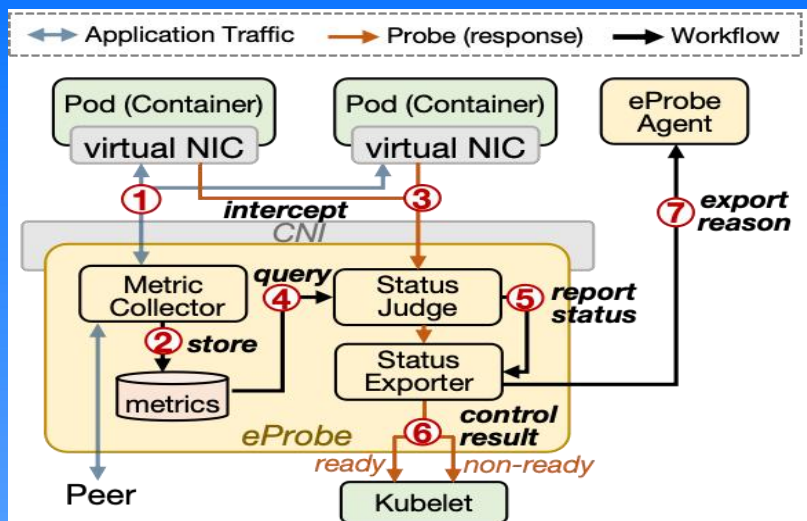


探测配置复杂

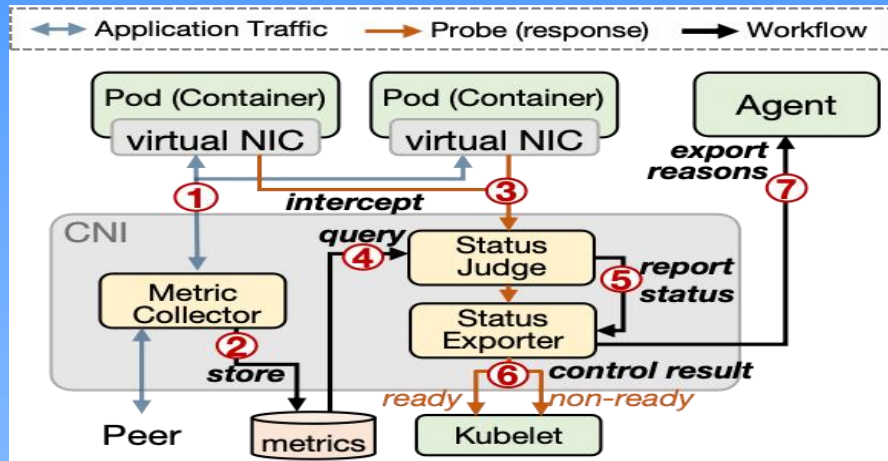
需为每个容器手动配置探针
不同应用需不同探针类型
配置错误是故障主因之一

eBPF增强的容器探测架构

基于TC层非侵入采集 + 拦截Kubelet探针动态改写响应



eBPF增强容器探测架构: Metric Collector + Status Judge + Status Exporter



三大模块

1 Metric Collector

eBPF程序挂载于TC层
拦截容器入出站流量
收集响应率/响应时间/连接数/错误率
存储在eBPF Maps中

2 Status Judge

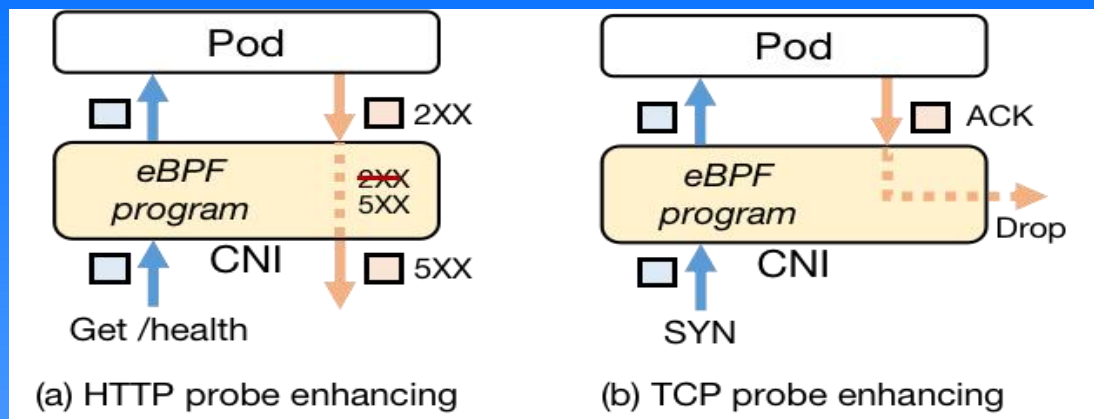
拦截Kubelet探测请求
从Maps读取性能指标
多维指标判断Readiness/Liveness
自适应阈值配置

3 Status Exporter

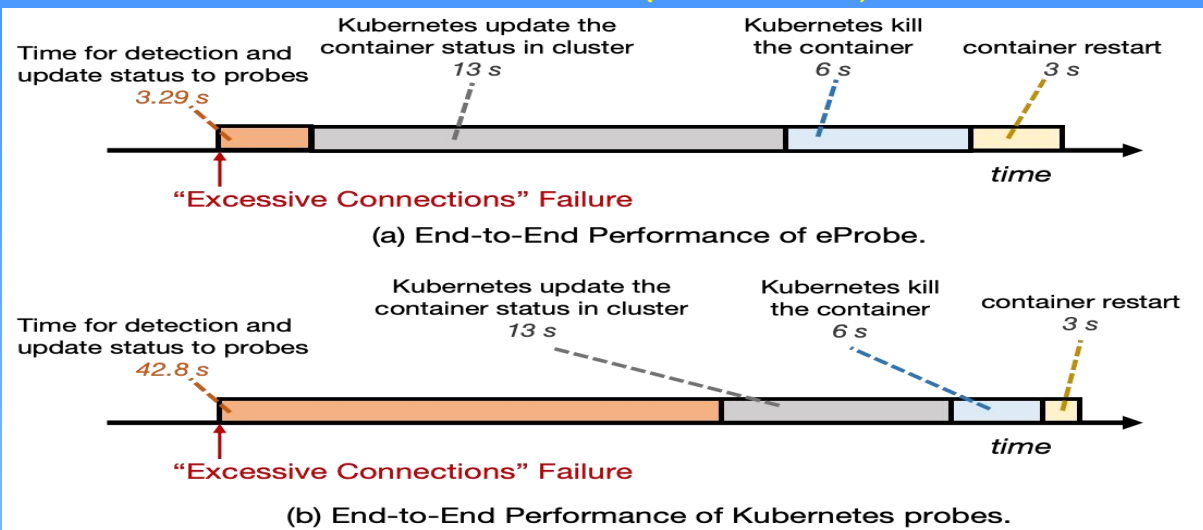
控制探测响应返回Kubelet
导出详细异常原因
提供细粒度诊断信息

探针增强与四维健康指标

拦截Kubelet探针请求 + eBPF实时指标动态改写响应



根据容器状态改写HTTP探针返回码 (2XX→5XX) / 丢弃TCP ACK



2026年4月 · 探测时序: Kubelet周期探测与eBPF指标实时对齐

四维容器健康指标

响应率 (Response Ratio)

HTTP/MySQL成功响应比例
低于阈值→服务降级

响应时间 (Response Time)

DDSketch分位数延迟统计
显著增长→性能瓶颈

连接数 (Connections)

活跃TCP连接数量
异常增长→资源泄漏

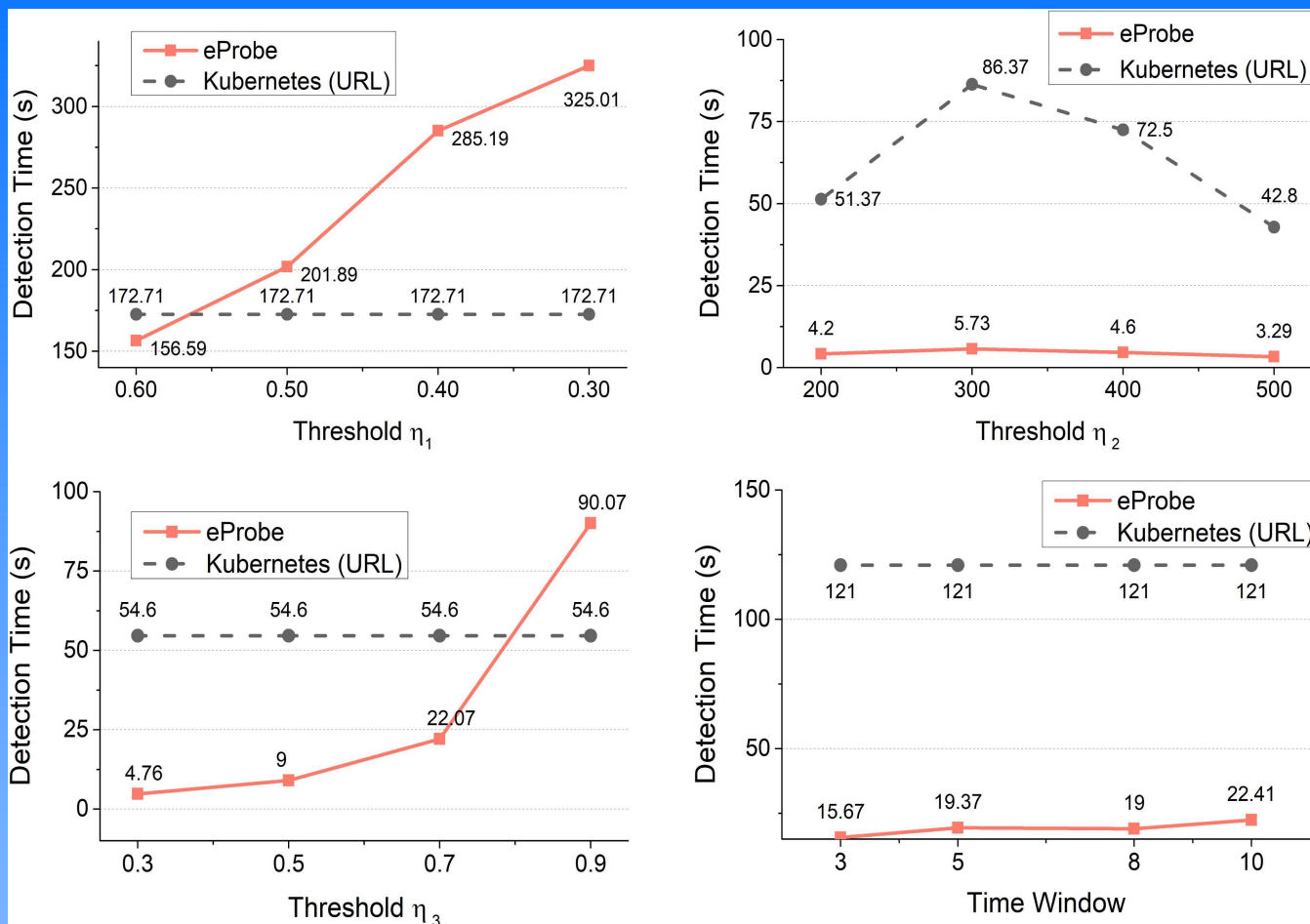
错误率 (Error Rate)

HTTP 4xx/5xx和MySQL错误
高错误→non-ready

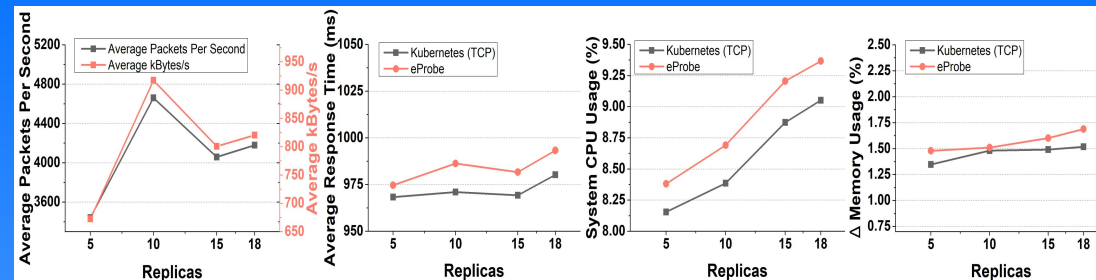


容器探测的实验评估

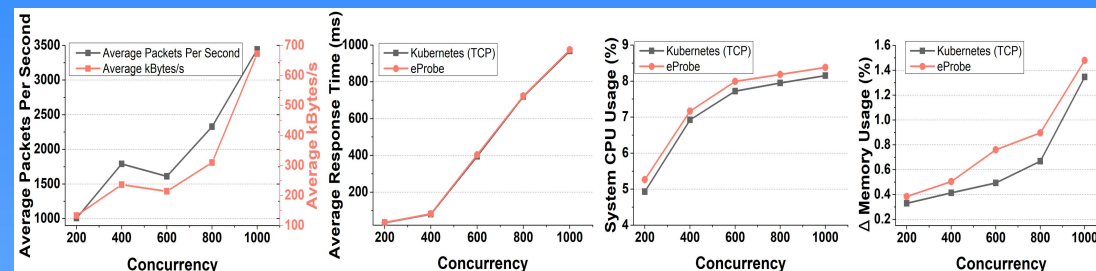
Sock-Shop七大故障场景 + 可伸缩性测试



eBPF探针 vs K8s探针: 7种故障场景检测能力对比



可伸缩性 (按实例数)



可伸缩性 (按用户数)

100%
异常检测率

2.48%
响应时间增加

0.33%
CPU额外开销

0.26%
内存额外开销

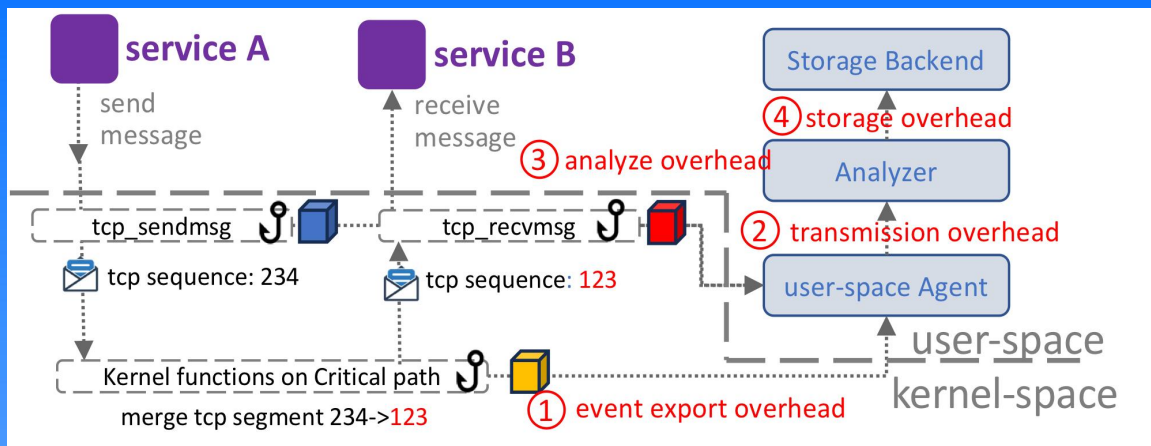
04 突破协议限制

HTTP追踪解决了, 但gRPC/Thrift无处不在
如何在保证准确率的同时, 可控地降低追踪开销?

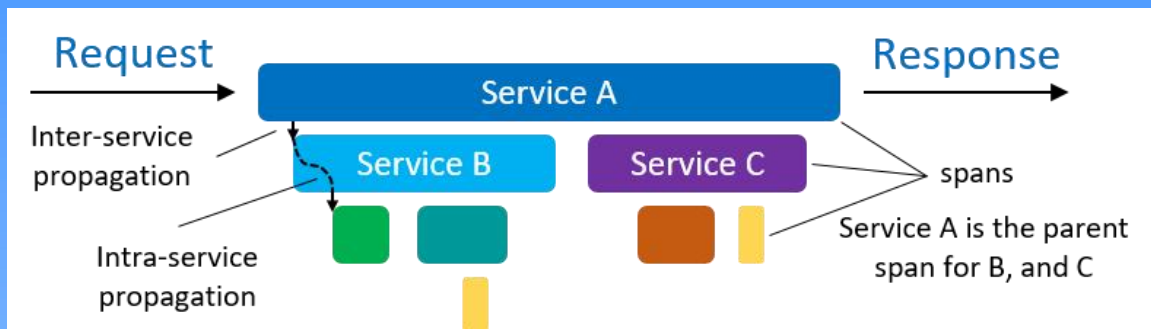
内核级多协议上下文传播 + Trace级采样

精度-开销的根本两难

非侵入追踪的四大设计目标



非侵入追踪的精度-开销权衡空间



分布式Trace示例: 跨服务span关系与执行时间

四大设计目标

Non-intrusiveness

完全不修改应用代码或第三方库
对开发者透明

Coverage (语言/协议无关)

支持HTTP/gRPC/Thrift等所有TCP协议
不区分编程语言/处理模型

Lightweight

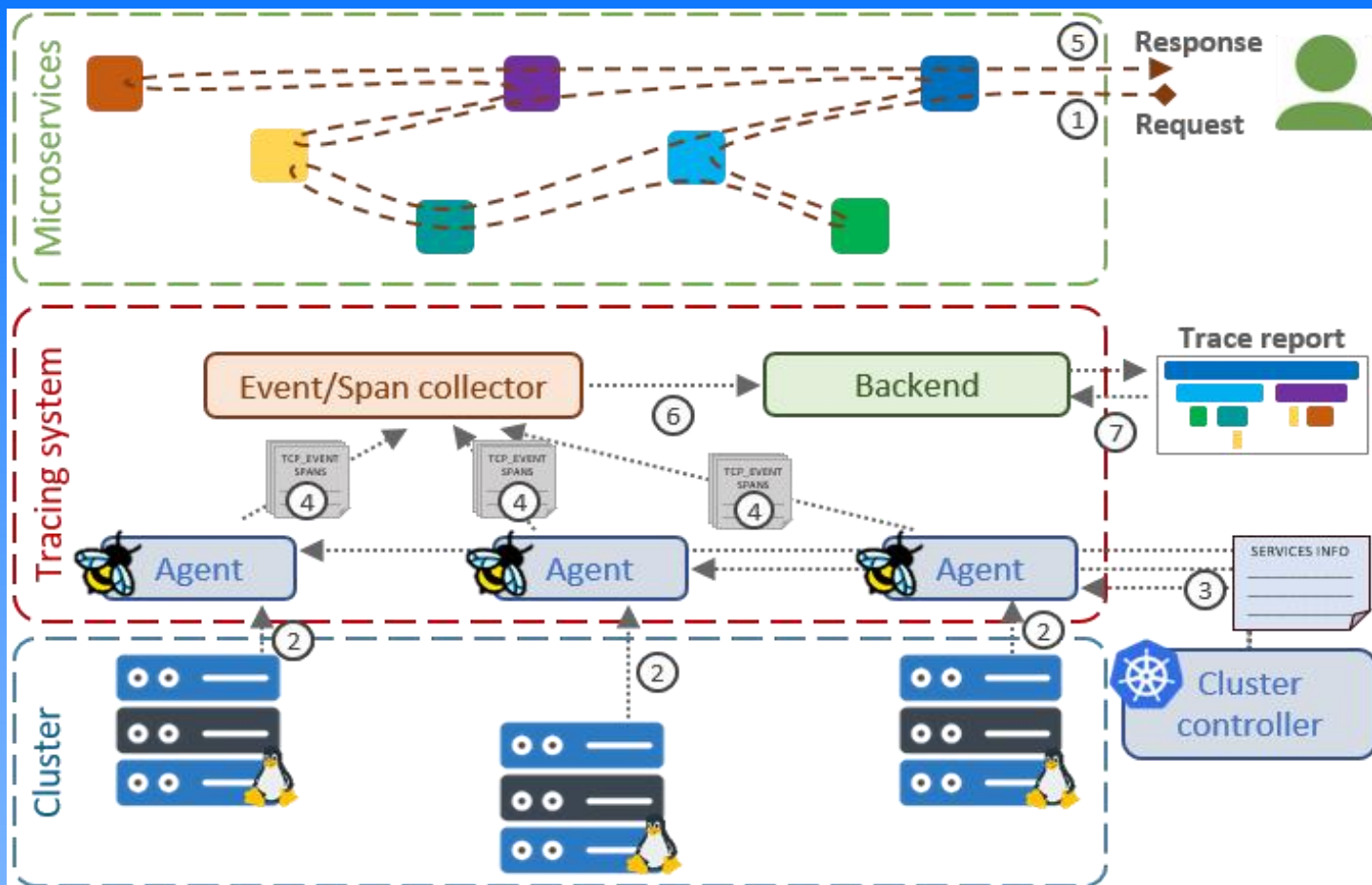
支持Trace级采样, 可调节精度-开销平衡
破除全量采集或随机丢弃的两难

Accuracy

在多协议、协程、RPC多路复用等
复杂场景下仍保持高准确率

多协议追踪：系统架构总览

分布式Agent + 集中式Collector



多协议追踪高层设计：每节点Agent + 集群Controller + Collector

三大组件

Cluster Controller

下发服务拓扑、采样率、
协议配置给各Agent
集中式管理策略

eBPF Agent (每节点)

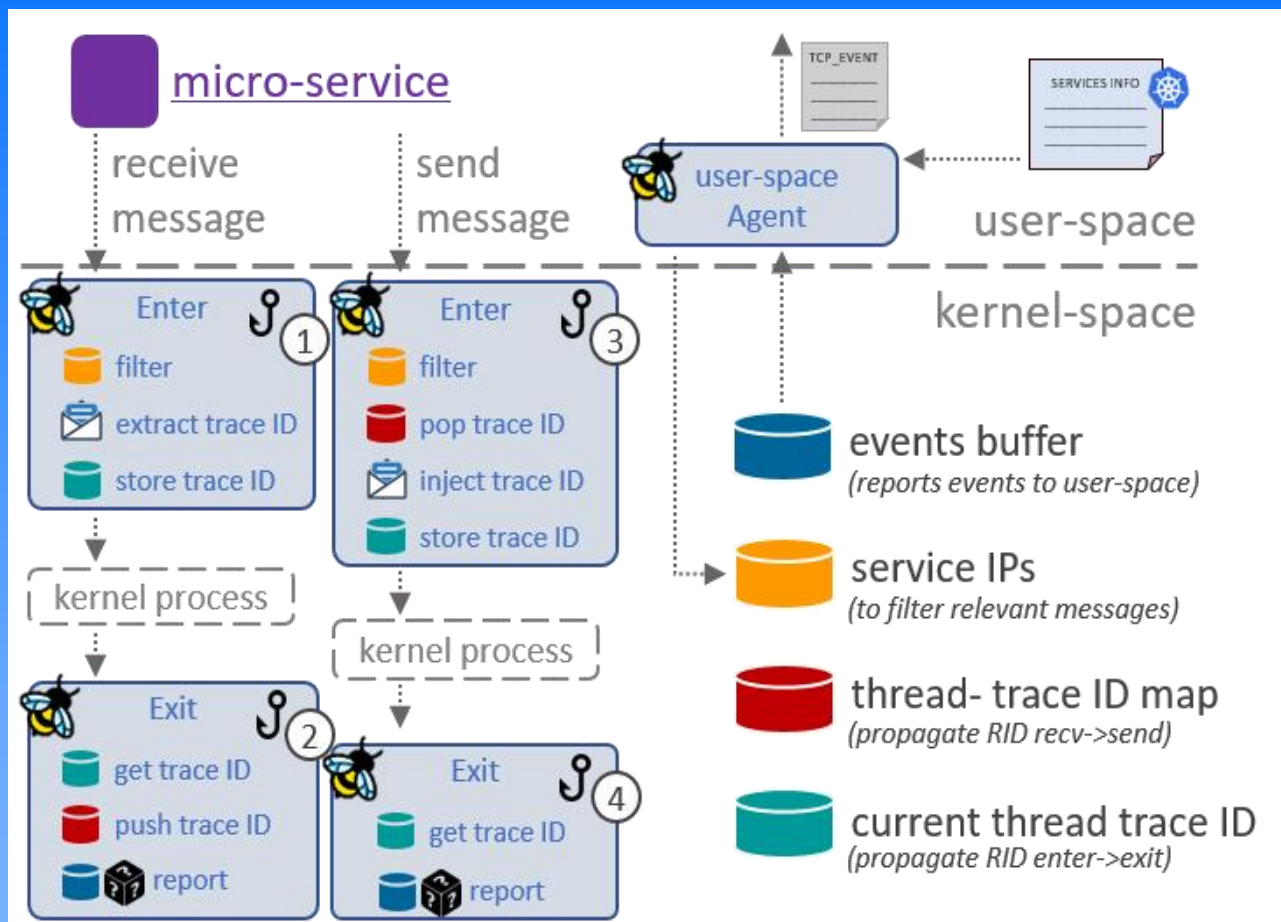
内核态eBPF程序
拦截/注入请求头部
执行Trace级采样决策

Data Collector

接收各Agent上报的Span
重建完整Trace
Service Tracer做拓扑推断

内核态Agent: 架构与四大设计决策

eBPF程序的Hooks放置、采样、打标、事件粒度



Agent内部: 接收/发送路径 eBPF Hooks + Maps + Ring Buffer

4个关键设计决策

① Hooks放置

TC层拦截, 语言无关
TCP send/recv两侧对称挂钩

② Sampling策略

Trace级: 基于Trace ID哈希
同一trace全采或全弃

③ Tagging机制

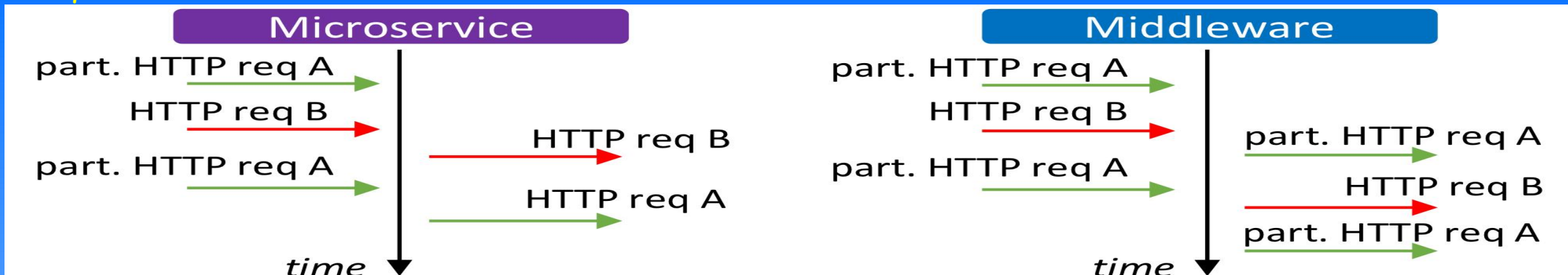
协议原生Header (HTTP/gRPC/Thrift)
不侵入应用语义

④ Events粒度

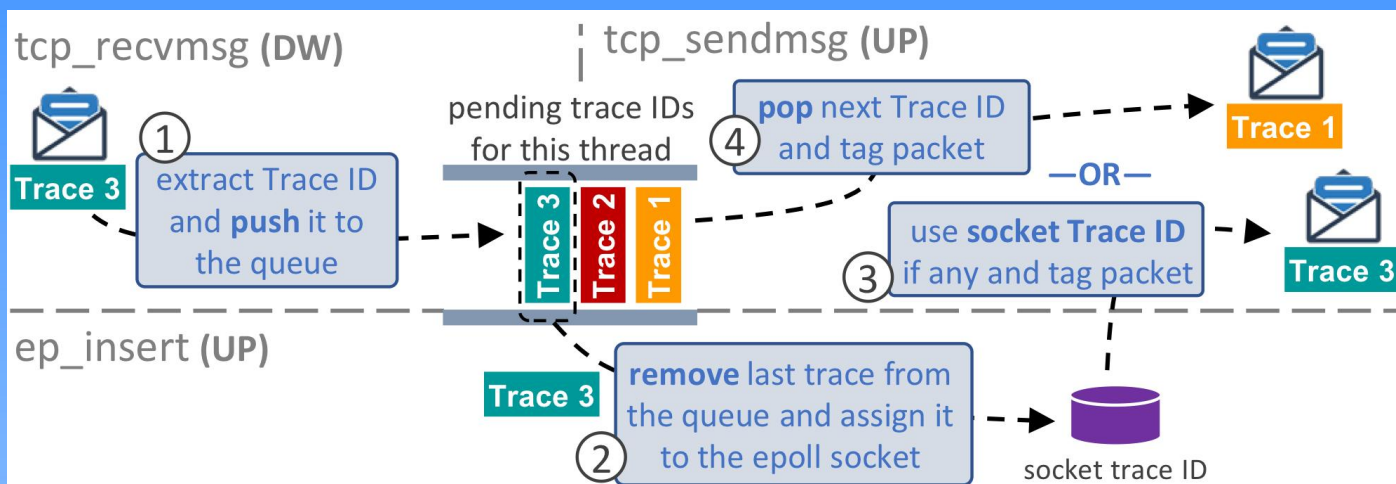
事件合并 + 批量上报
控制Ring Buffer开销

服务内上下文传播机制

epoll-enhanced FIFO: 应对中间件/代理复杂处理模型



微服务 vs 中间件 场景下的服务内上下文传播行为差异



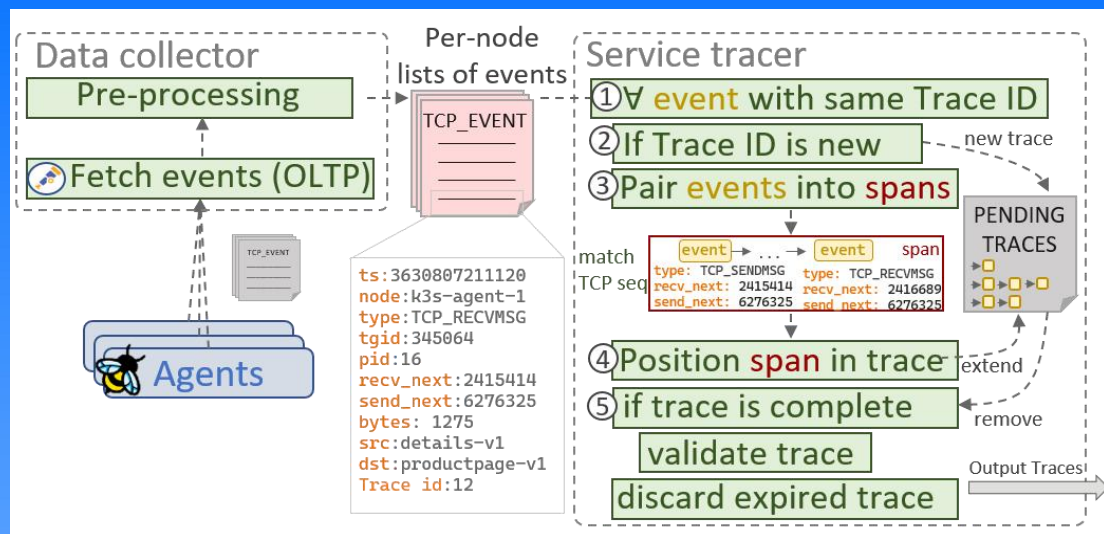
关键技术点

- ▶ 微服务: 1请求=1线程, 传播简单
- ▶ 中间件/代理: 接收线程 ≠ 处理线程
- ▶ *FIFO-only*: 跨线程误关联
- ▶ *FIFO + epoll*: 精确跟踪工作交接
- ▶ 仅带来微小额外开销

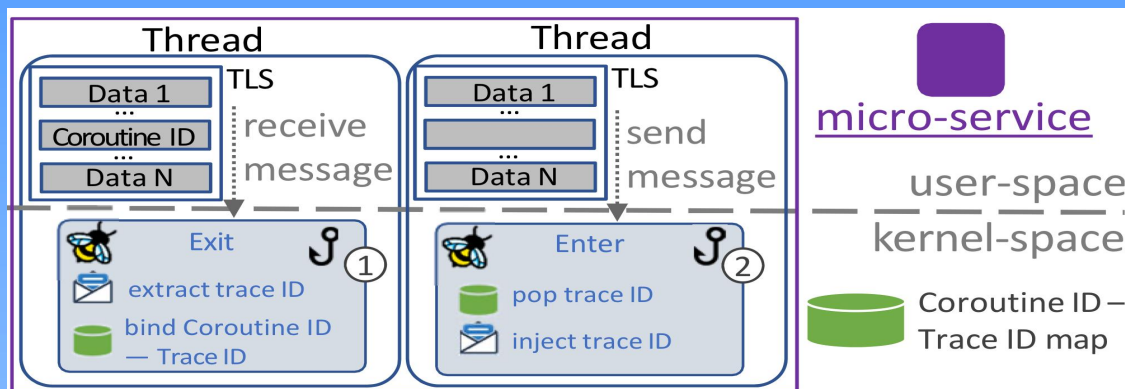
Epoll-enhanced FIFO: 利用*epoll*事件精确关联接收与处理线程

Trace重建与高级扩展

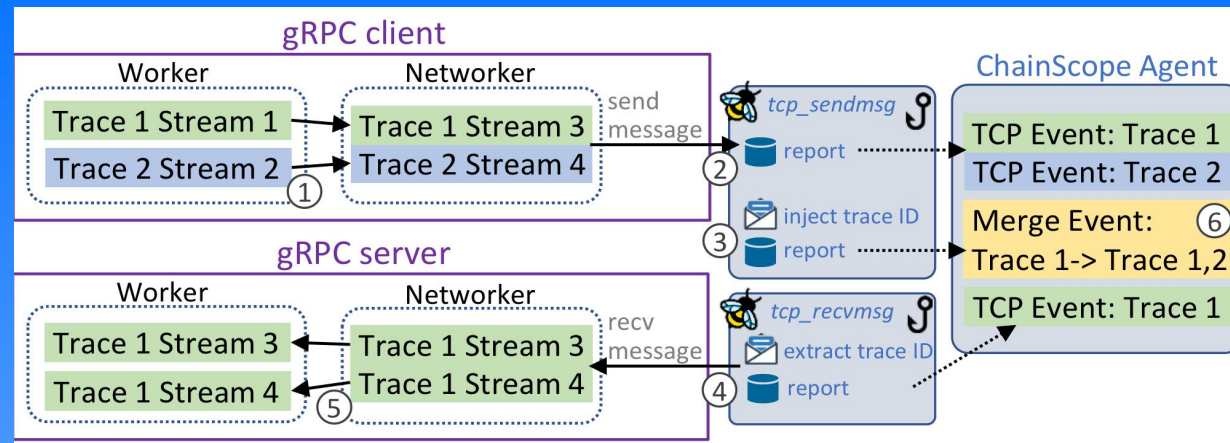
Collector拓扑重建 + 协程/gRPC多路复用支持



Collector: Data Collector + Service Tracer 双流水线



2026年4月 · 中国 协程支持: Goroutine服务内上下文传播



gRPC: 解析HTTP/2 HEADERS帧, 支持stream ID多路复用

两大扩展场景

协程 (Goroutines)

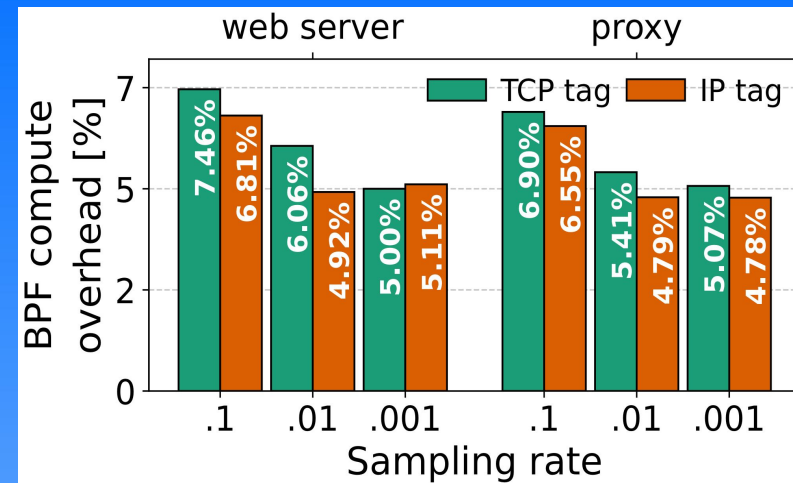
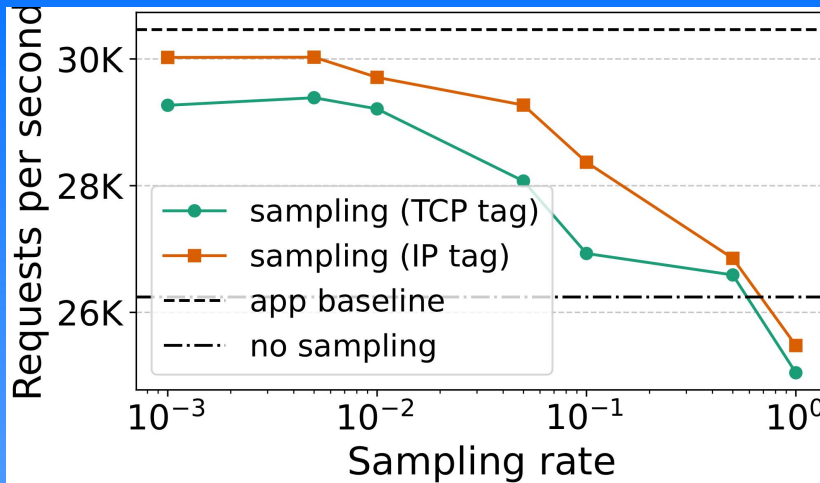
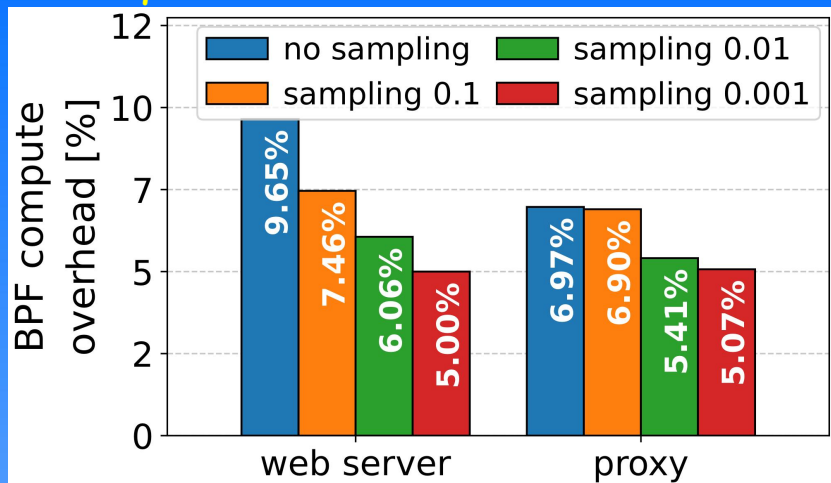
kprobe跟踪go scheduler事件
建立goroutine → trace context映射

gRPC (HTTP/2多路复用)

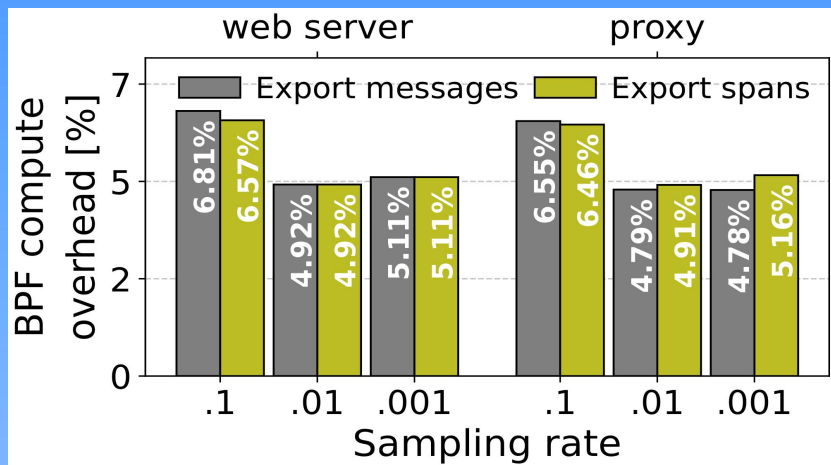
按stream ID维度传播trace context
单TCP连接多stream并行

微基准验证：四大设计决策

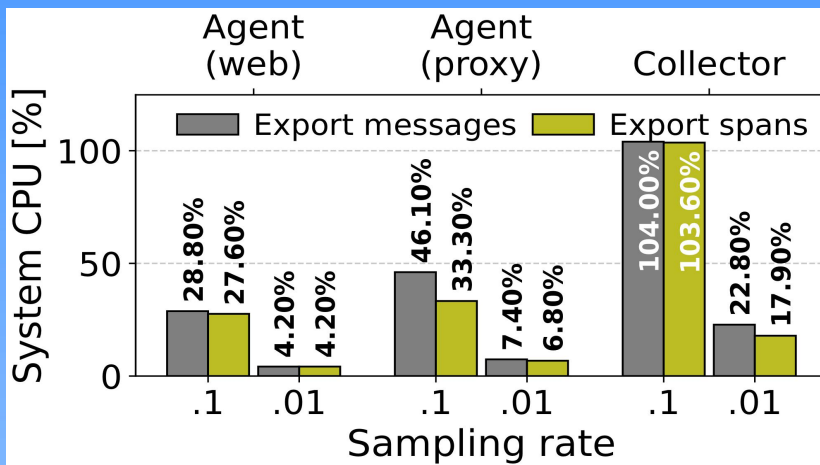
Apache Benchmark 32并发请求



① 采样: eBPF CPU占比



① 采样: E2E吞吐量 (QPS)



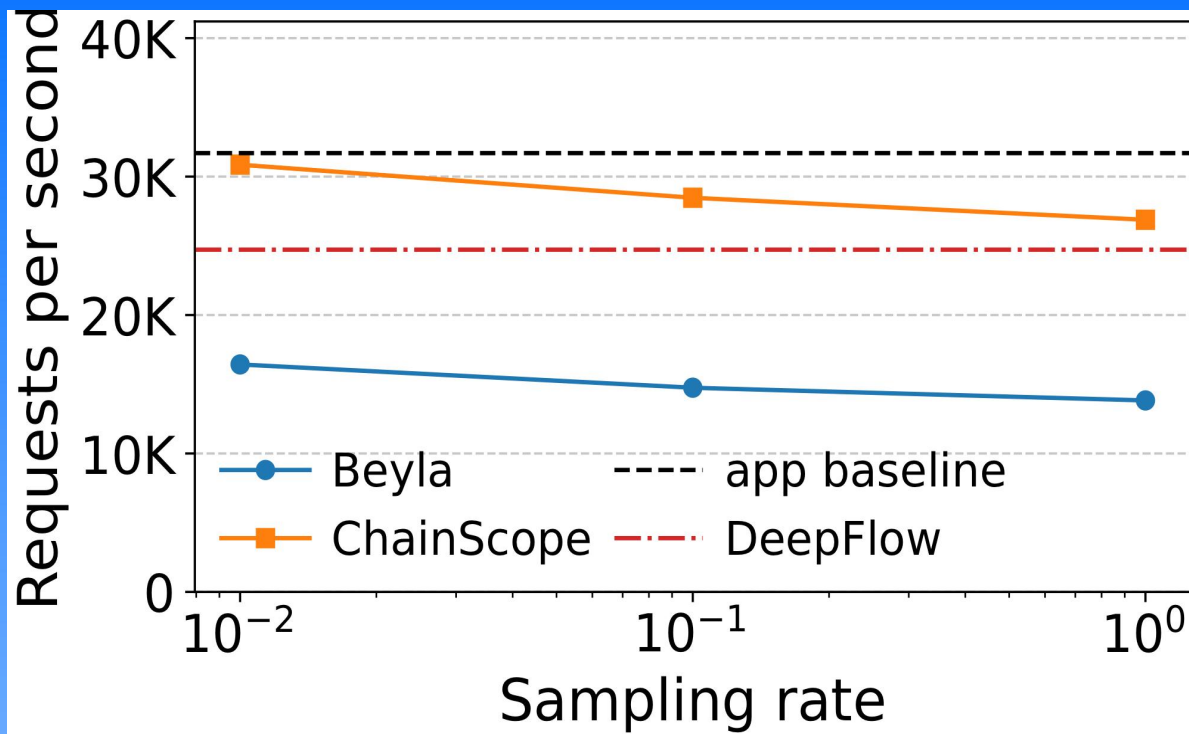
③ 打标协议: eBPF开销

结论

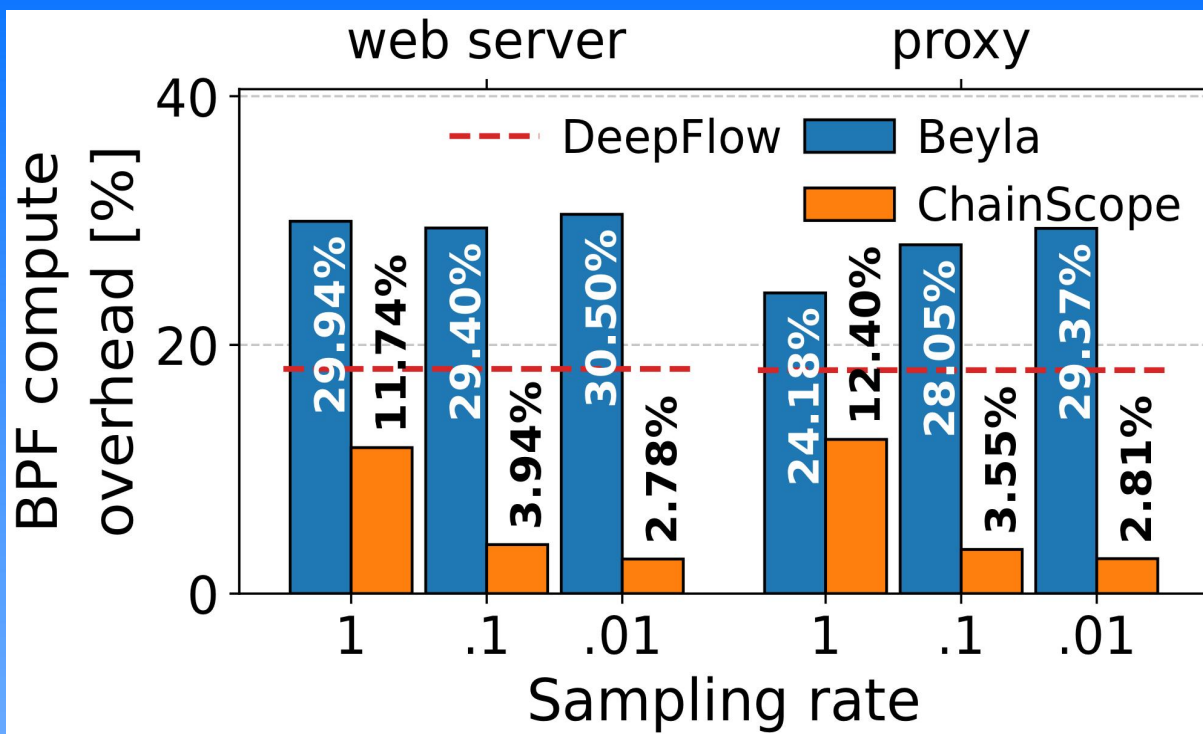
- ▶ 采样率↓ → 开销近线性下降
- ▶ HTTP header tagging几乎零成本
- ▶ 批量事件上报比逐事件省50%+
- ▶ 可灵活调节精度/开销平衡

多协议追踪评测 (一): Nginx基线场景

Q1/Q2/Q4: 重请求负载下的吞吐量与CPU开销对比



端到端链路吞吐量 (RPS)



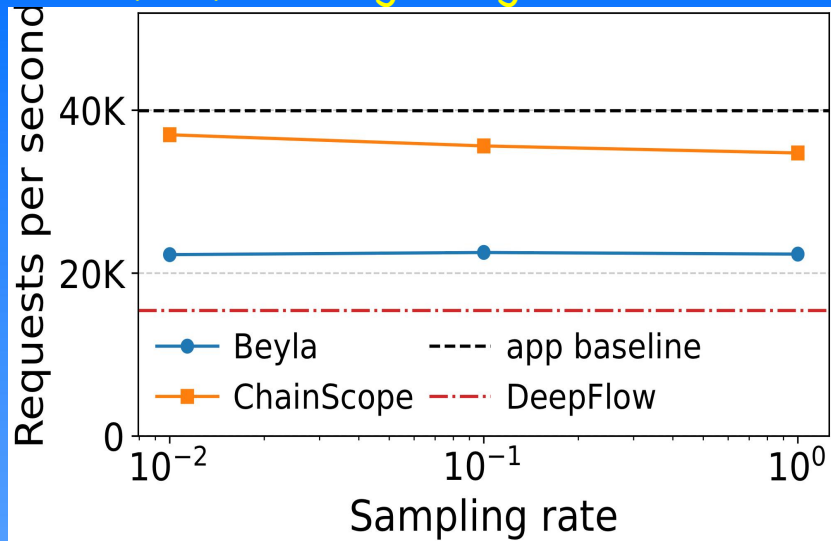
追踪系统CPU开销 (eBPF程序)

关键观察

- ▶ ChainScope全采样下仍保持与DeepFlow相当的吞吐
- ▶ 10%采样率下开销比DeepFlow低约40%
- ▶ Trace级采样不影响trace完整性
- ▶ 同样10%采样率: ChainScope保留trace完整率 > 99%, 传统packet采样 < 15%

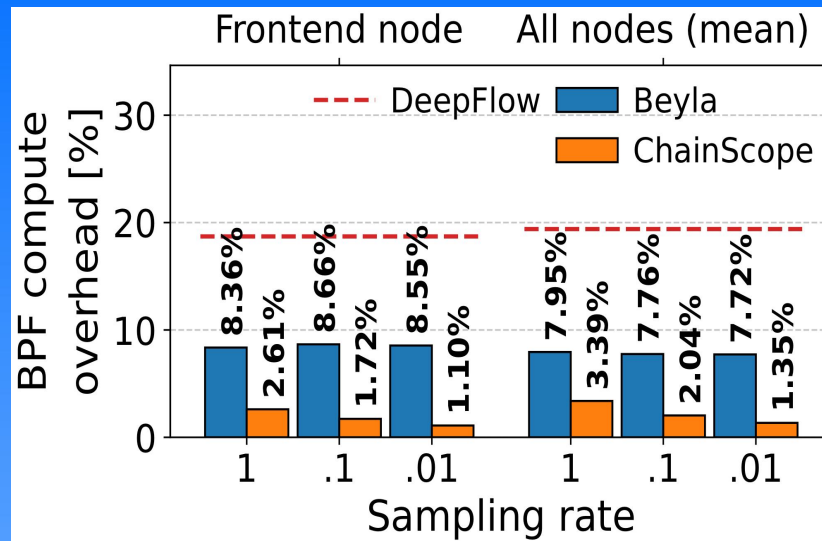
多协议追踪评测 (二): gRPC复杂场景

Q3/Q4: 9服务gRPC+goroutine mesh下的准确率与吞吐



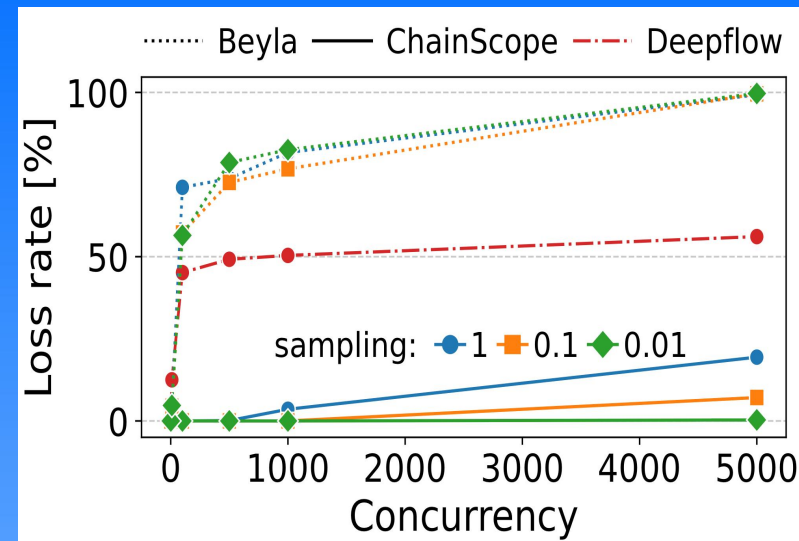
E2E吞吐量 (9服务mesh)

2.2x
准确率提升
vs DeepFlow



eBPF Agent CPU开销

1.6x
吞吐量提升
gRPC场景



gRPC追踪丢失率

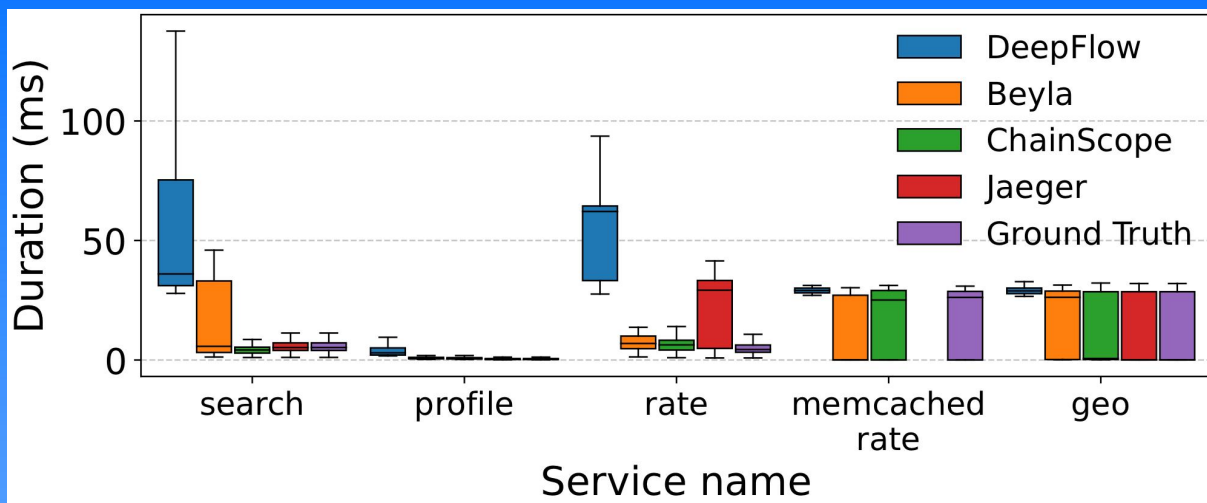
HTTP/gRPC/Thrift
多协议统一

核心结论: 在9服务gRPC mesh+协程并发下, ChainScope同时实现'最高准确率'和'最低开销'

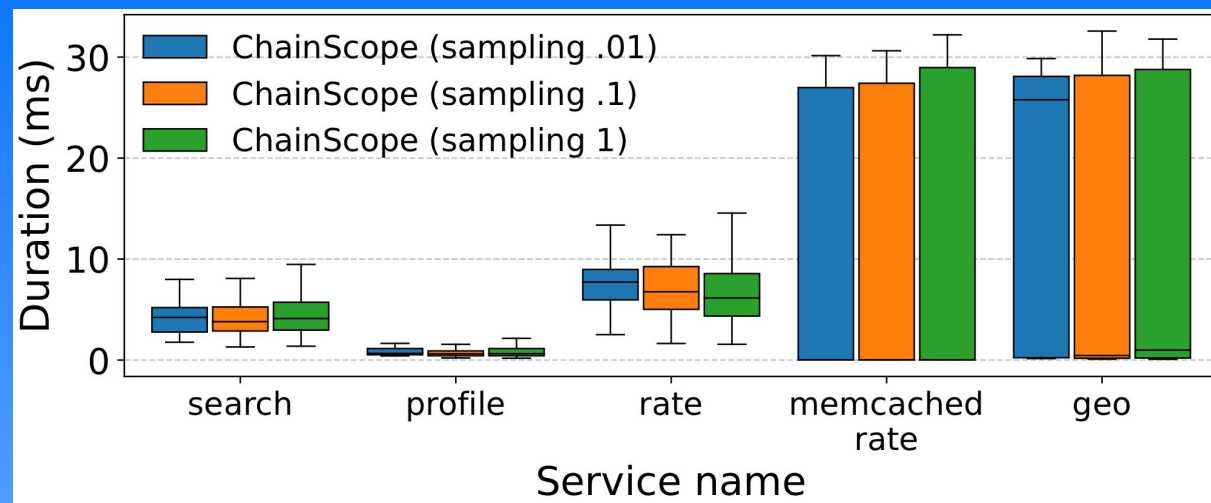
打破了非侵入方案的精度-开销两难, 且trace完整性远超packet级采样方案

根因分析实战 (Q5)

注入延迟故障后, 追踪系统能否精准定位根因?



各追踪系统在根因分析任务下的准确率



不同采样率下根因分析的稳定性

四步根因分析 workflow

1 异常检测

P99延迟异常飙升

2 Trace检索

取异常时段完整Trace

3 Span分析

对比正常/异常延迟

4 根因定位

精确到服务+操作

05 向上延伸: GPU计算图

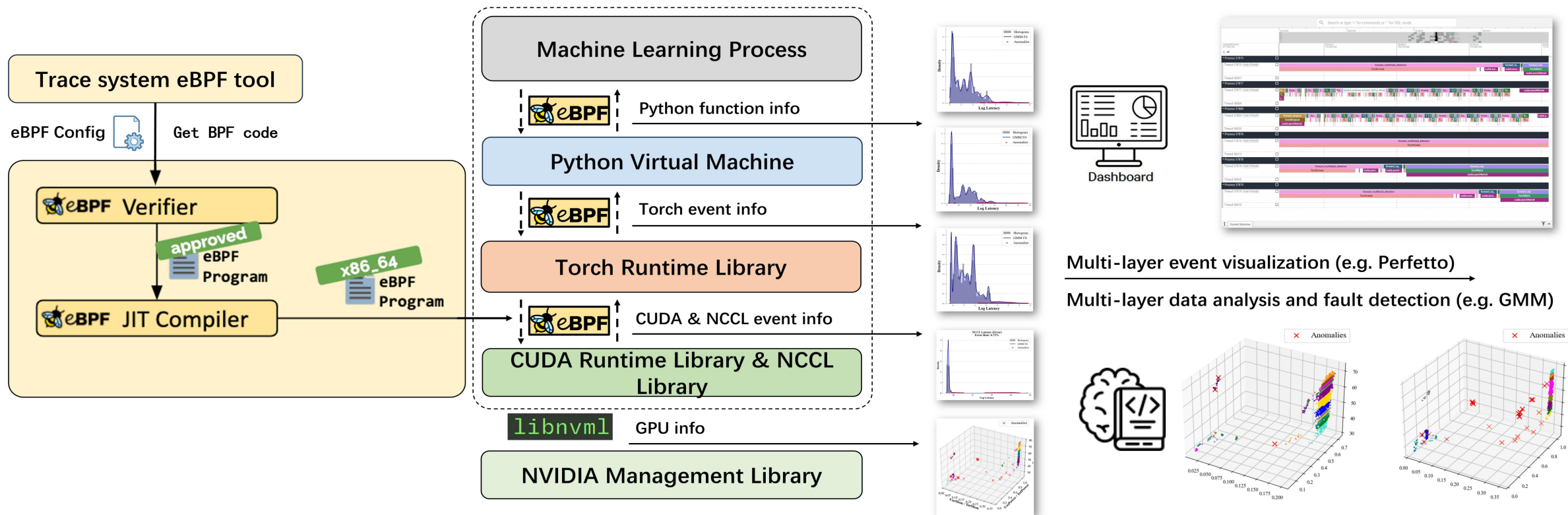
微服务追好了, 但大模型时代来了——GPU上的计算模式谁来追踪?

eBPF赋能全栈AI/ML系统可观测性



GPU计算图追踪: 全栈架构

*eBPF + libnvml + GMM*异常检测, 跨软硬件栈的非侵入监控



GPU计算图追踪架构: 跨CUDA/Python/PyTorch软件层 + GPU/N CCL硬件层

核心: eBPF uprobe拦截CUDA API + libnvml进程级GPU指标 + GMM建模异常检测

GPU追踪的核心技术创新

CUDA/Python/PyTorch拦截

- ▶ eBPF uprobe拦截 `cudaLaunchKernel` 等API
- ▶ Python `bytecode`层拦截训练循环
- ▶ 无需修改框架或训练代码

libnvmf进程级GPU监控

- ▶ 进程级GPU利用率/显存/功耗实时采集
- ▶ 识别单/多进程GPU共享瓶颈
- ▶ 比`nvidia-smi`更细粒度

GMM异常检测

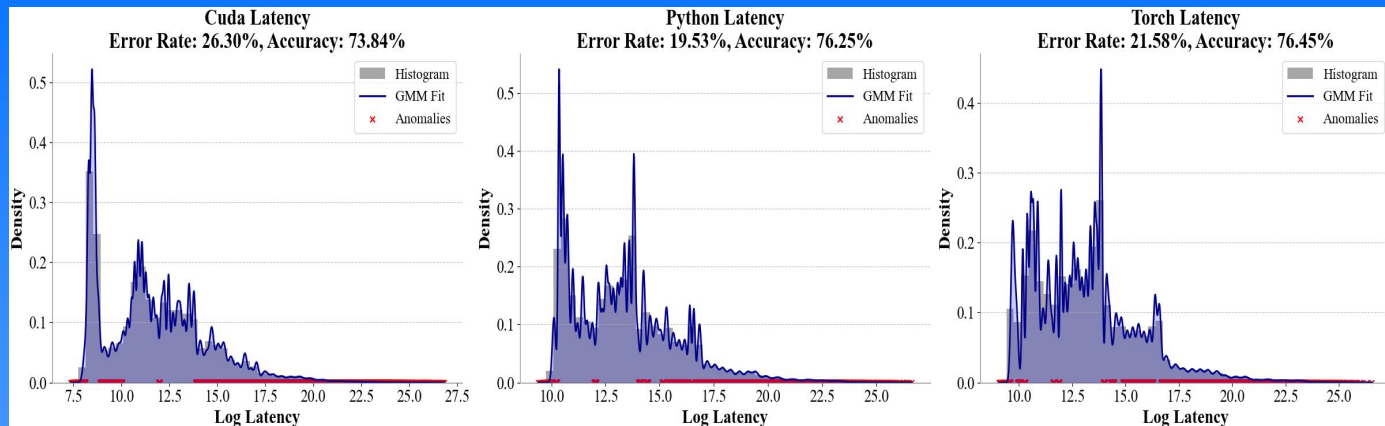
- ▶ 多维性能指标混合高斯建模
- ▶ 自动识别延迟/硬件/通信三类异常
- ▶ 无需人工设定阈值

分布式训练诊断

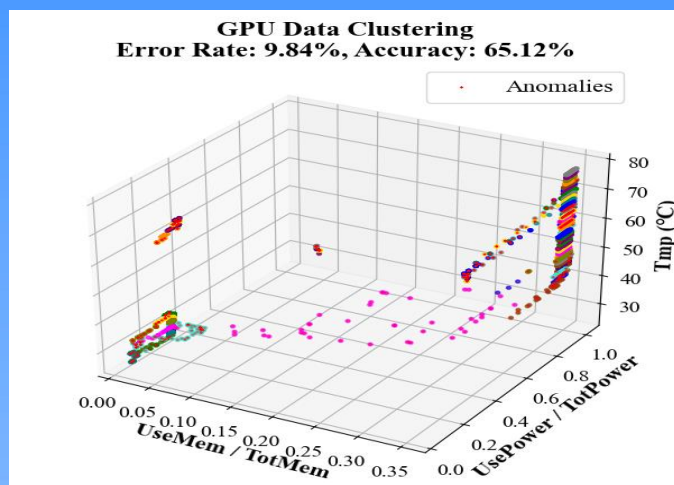
- ▶ NCCL `AllReduce/Broadcast`通信追踪
- ▶ 多节点性能瓶颈定位
- ▶ 支持LLM训练典型场景

GPU追踪的实验结果

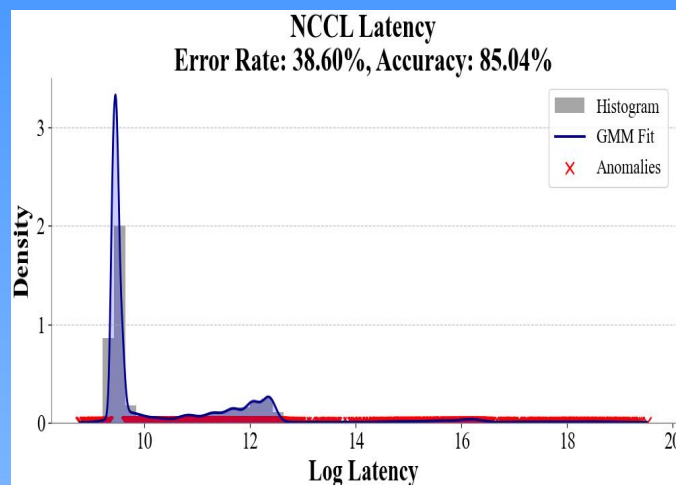
多节点分布式训练, 三类异常GMM检测



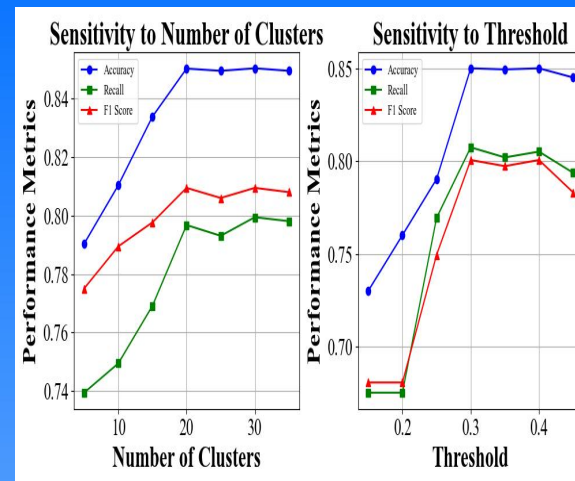
延迟异常检测: CUDA 73.84% / Python 76.25% / Torch 76.45%



硬件异常 65.12%



通信异常 85.04%



GMM 敏感性分析

85.04%
通信异常
(最佳)

76.45%
延迟异常
(Torch层)

6类
对比基线
(KMeans等)

应用场景

- ▶ LLM训练: 追踪kernel调度, 识别通信瓶颈
- ▶ 推理监控: GPU利用率回退, 资源竞争
- ▶ 分布式训练: AllReduce/NCCL瓶颈定位

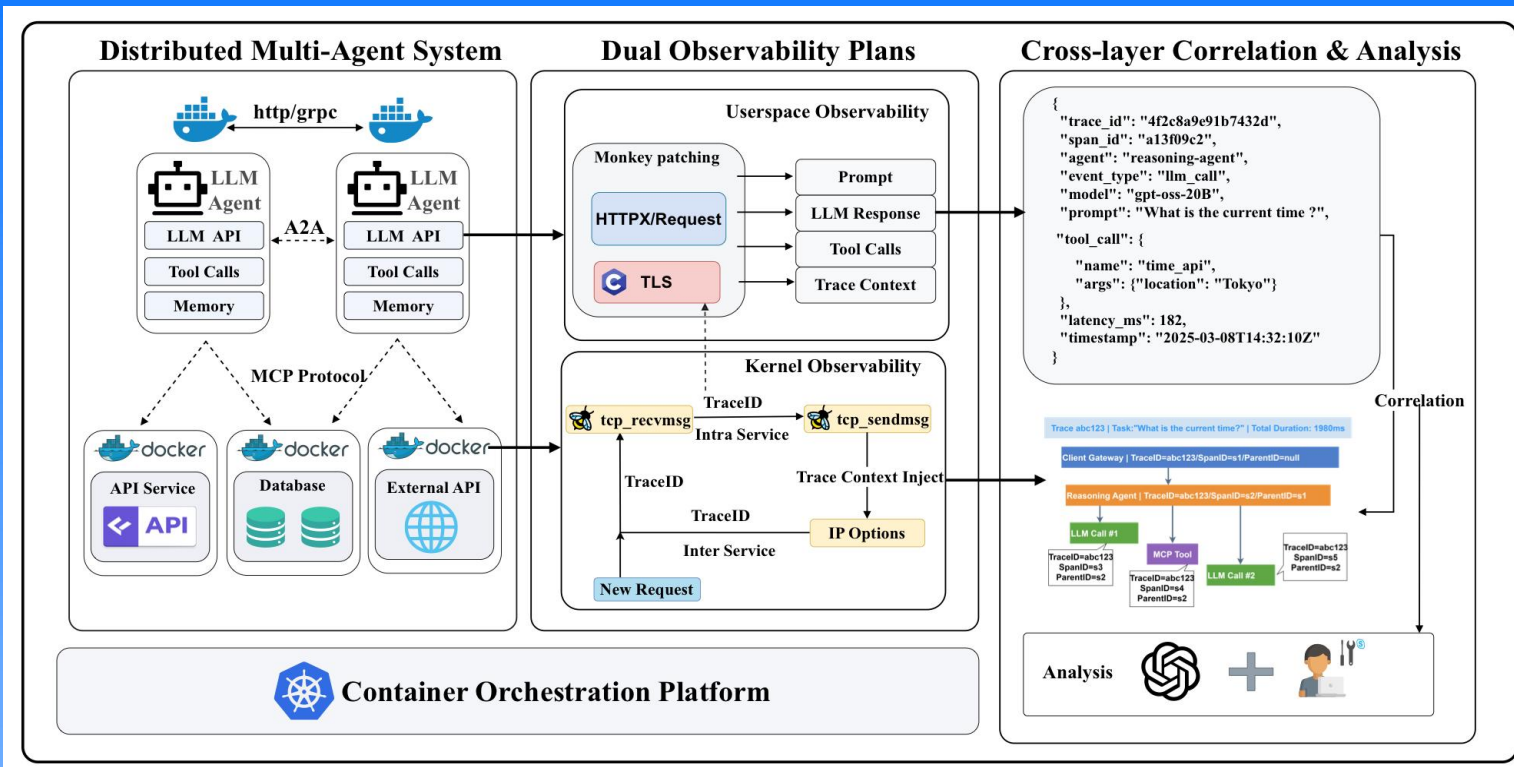
06 AI时代: 多智能体追踪

Agent调用Agent, Agent又调用工具——这条链路, 谁来追踪?

从云原生eBPF追踪走向AI Agent可观测性

多智能体追踪：跨层架构

内核eBPF捕获基础设施 + 运行时Hook捕获Agent语义



多智能体追踪整体架构

核心创新

观测鸿沟

LLM工具只懂语义
TCP追踪只懂基础设施
两者都要代码改动

方案

IP层eBPF传播trace身份
协议/语言无关
运行时hook捕获Agent语义

难点

asyncio并发关联
跨同步/异步runtime
确定性因果推断

效果

<4%延迟, <5% CPU
9类故障场景全覆盖

Agent追踪模型: 分层Span定义

用户Query → 多服务DMAS工作流的层次化因果分解

Trace abc123 | Task:"What is the current time?" | Total Duration: 1980ms

Client Gateway | TraceID=abc123/SpanID=s1/ParentID=null

Reasoning Agent | TraceID=abc123/SpanID=s2/ParentID=s1

LLM Call #1

TraceID=abc123
SpanID=s3
ParentID=s2

MCP Tool

TraceID=abc123
SpanID=s4
ParentID=s2

LLM Call #2

TraceID=abc123
SpanID=s5
ParentID=s2

跨3服务DMAS工作流的trace模型: 层次化span关系 + 因果分解

Agent Span类型

Agent Span

完整生命周期
思考→决策→执行

LLM Span

一次LLM API调用
Prompt→Token→响应

Tool Span

外部工具执行
搜索/代码/数据库

Communication

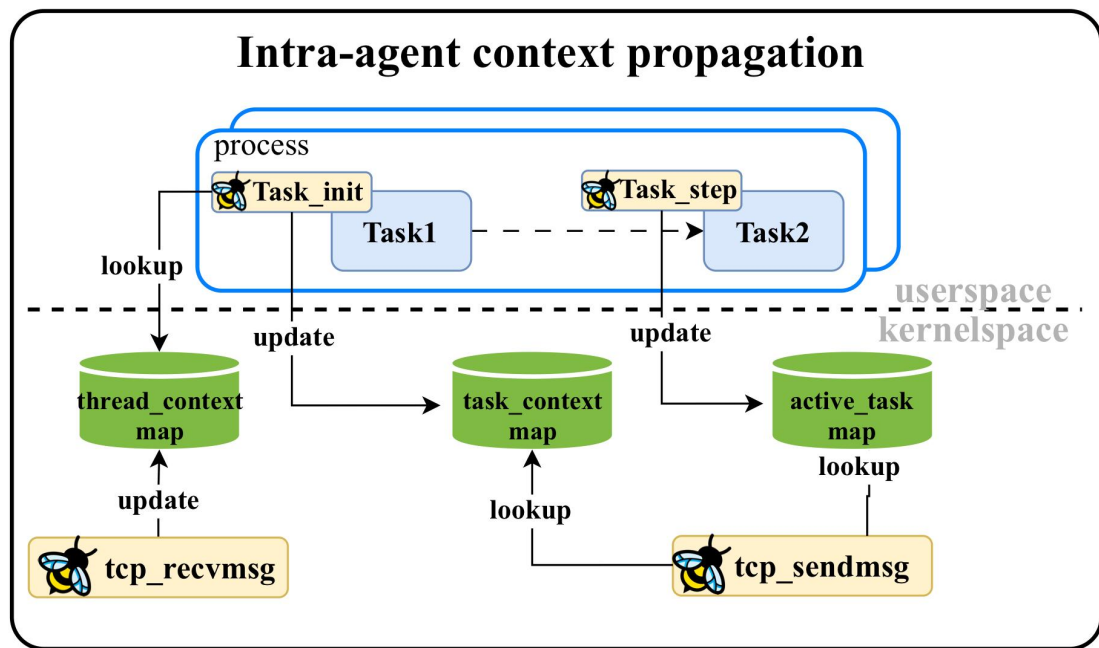
Agent间消息传递
任务委托/结果返回

Memory Span

记忆系统交互
RAG/上下文检索

多智能体追踪：关键技术与评测

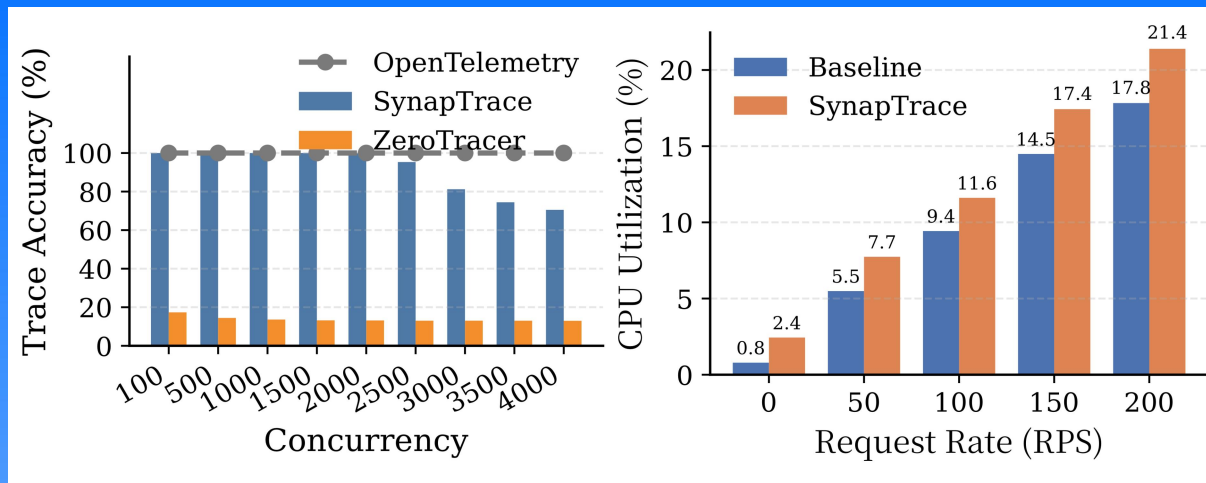
asyncio并发关联破解 + 100% trace准确率, <5% CPU开销



三段中继: thread-anchored → task-relay → deterministic retrieval

关键技术突破

- 解决 asyncio 协程 'correlation collapse' 问题
- 同/异步runtime确定性因果关联 (vs ZeroTracer仅13-17%)
- K8s 3个DMAS部署验证, 语义+基础设施+跨层故障全覆盖



Trace准确率 (按Tool)

CPU利用率开销

100%
Trace准确率
(ZeroTracer仅13-17%)

<4%
端到端延迟开销

<5%
CPU开销
(平均1.65-3.56%)

9类故障
语义/基础设施/
跨层全覆盖

五个阶段：从微服务到多智能体的追踪演进

Stage 1

HTTP请求追踪

零侵入
内核态生成完整
Trace
支持多线程服务

Stage 2

容器健康感知

*eBPF*增强*K8s*探针
四维健康指标
实时状态判断

Stage 3

多协议精确追踪

HTTP/gRPC/Thrift
内核上下文传播
*Trace*级采样

Stage 4

*GPU*计算图追踪

CUDA API拦截
计算图构建
*GMM*异常检测

Stage 5

多智能体追踪

*LLM*调用链路
*Agent*语义捕获
*asyncio*关联

核心理念: *eBPF*作为统一的内核态可观测性基础设施

从微服务 → 容器 → 多协议 → *ML*系统 → 多智能体, *eBPF*持续向上层应用栈延伸

07 总结与展望

*eBPF*驱动的端到端可观测性

*eBPF*不只是追踪工具，它正在成为AI时代系统可观测性的神经系统



研究总结

HTTP追踪
TPDS'25

首个零侵入eBPF分布式追踪生成器
91%+准确率, 0.5-1.2%延迟开销

91%+准确率, 0.5-1.2%延迟开销

容器探测
TSC'25

eBPF增强容器健康状态精准感知
100%异常检测, 0.33%CPU开销

100%异常检测, 0.33%CPU开销

多协议追踪
CoNEXT'26

内核级上下文传播+Trace级采样
2.2x准确率, 1.6x吞吐量提升

2.2x准确率, 1.6x吞吐量提升

GPU追踪
IWQoS'25

非侵入GPU计算图追踪与异常检测
85%通信异常, GMM建模

85%通信异常, GMM建模

Agent追踪
ASE'26投稿

多智能体系统端到端追踪
100% trace准确率, <5% CPU

100% trace准确率, <5% CPU

统一主线: eBPF内核态可编程 → 零侵入 → 端到端可观测性 → 从微服务到AI Agent

未来展望



更广的协议覆盖



QUIC、WebSocket、
Kafka/RabbitMQ
等新兴通信协议的非侵入追踪



AI原生可观测性



LLM推理链、RAG管道、多
Agent协作
MoE路由的专用追踪框架



智能化根因分析



结合LLM进行自动化Trace分析
根因推理和修复建议生成



跨层统一追踪

应用→容器→内核→硬件
(GPU/NPU)
全栈可观测性闭环



生态深度集成

与
OpenTelemetry/Prometheus/
Grafana
CNCF生态深度融合



开源社区建设

推动核心技术开源
构建学术-工业桥梁



第四届eBPF开发者大会

谢谢!

Thank You!

陈鹏飞 | 中山大学计算机学院

chenpf7@mail.sysu.edu.cn

GitHub: github.com/IntelligentDDS